

SalesVerse 2.0

Solution Architecture

Application, data, integration, security, deployment and infrastructure architecture

IIFT Agent/Banca Portal & Back-office for Insurans Islam Family Takaful Sendirian Berhad



PREPARED FOR

Insurans Islam Family Takaful Sendirian Berhad

Bandar Seri Begawan, Brunei Darussalam

PREPARED BY

iorta TechNXT

<https://iortatechnxt.com>

Document reference	IIFT-SV2-SAD
Version	1.0
Date	October 2026
RFP deliverable	DEL-06 (Solution Architecture)
Status	Issued with the proposal
Classification	Commercial-in-Confidence

Commercial-in-Confidence. Prepared by iorta TechNXT for Insurans Islam Family Takaful Sendirian Berhad as part of the response to IIFT's Request for Proposal for an Agent/Banca Portal and Back-office solution. Not to be disclosed to third parties without prior written consent.

Document Control

This document is the Solution Architecture for SalesVerse 2.0 as configured for IIFT. It is maintained throughout the contract and updated after every material change (NFR-21, MNT-27).

Version history

Version	Date	Author	Description
0.9	September 2026	iorta TechNXT	Internal technical review
1.0	October 2026	iorta TechNXT	Issued with the tender proposal

Review and approval

Role	Name	Signature / date
Prepared by (Solution Architect, iorta TechNXT)	[Name]	
Reviewed by (Engineering Lead, iorta TechNXT)	[Name]	
Approved by (Project Director, iorta TechNXT)	[Name]	
Accepted by (IIFT IT)	[Name, designation]	

Document pack

Reference	Document	RFP deliverable	Status
IIFT-SV2-SAD	SalesVerse 2.0 Solution Architecture	DEL-06	This document
IIFT-SV2-DD	SalesVerse 2.0 Data Dictionary	DEL-07	Issued
IIFT-SV2-PSH	SalesVerse 2.0 Production Support Handover	DEL-26, DEL-27, MNT-28	Issued
IIFT-SV2-CSQ	SalesVerse 2.0 Code Standards & Quality Report	DEL-11, NFR-14	Issued
IIFT-SV2-SAR	SalesVerse 2.0 Security Assessment Report	DEL-17, NFR-13	Issued

Related material

- SalesVerse-2.0-Data-Dictionary (.docx and .xlsx): every table and column.
- SalesVerse-2.0-Production-Support-Handover: support model, runbooks and configuration reference.
- docs/api/openapi.json: API description generated from the code.

Conventions

- Monetary amounts are in Brunei dollars (B\$). Times are Brunei time (UTC+8) unless marked UTC.
- RFP requirement identifiers (AP, BO, INT, COM, NFR, DEL, MNT) refer to IIFT's Request for Proposal. COM-01 to COM-10 in this pack are the shared platform requirements of RFP section 4.4.
- Items marked **delivered during implementation** are designed here but not yet present in the working application; everything else describes the application as it runs today.
- Text in [square brackets] is to be completed with IIFT during mobilisation.

Contents

Document Control.....	2
Contents.....	3
1. Purpose and Scope.....	5
1.1 Purpose.....	5
1.2 Scope.....	5
1.3 Audience.....	5
1.4 Status of the solution.....	5
1.5 Sources.....	6
2. Architecture Principles.....	7
3. System Context.....	8
4. Functional Architecture.....	10
4.1 Policy lifecycle.....	11
4.2 Maker-checker workflows.....	12
4.3 Scheduled processing.....	13
5. Logical and Application Architecture.....	14
5.1 Layers.....	14
5.2 Front end.....	14
5.3 API.....	15
5.4 Request processing.....	15
5.5 Error handling.....	15
5.6 Consistency and concurrency.....	16
6. Data Architecture.....	17
6.1 Entity overview.....	17
6.2 Why PostgreSQL.....	18
6.3 Integrity controls.....	18
6.4 Audit trail.....	18
6.5 Field-level encryption.....	19
6.6 Reference numbers.....	19
6.7 Documents.....	19
6.8 Configuration data.....	19
6.9 Retention and archival.....	20
6.10 Data migration.....	20
7. Integration Architecture.....	22
7.1 Approach.....	22
7.2 Delivery guarantees.....	22
7.3 Outbound operations.....	23

7.4 Inbound system APIs.....	23
7.5 End-of-day and reconciliation.....	24
7.6 Simulated and live modes.....	24
8. Security Architecture.....	25
8.1 Control catalogue.....	25
8.2 Roles.....	26
8.3 Keys and secrets.....	27
8.4 Security testing.....	27
9. Deployment Architecture.....	28
9.1 Environments.....	28
9.2 Option A: on-premise high availability (recommended).....	28
9.3 Option B: cloud.....	29
9.4 Option C: hybrid.....	30
9.5 Disaster recovery.....	31
9.6 Container images.....	31
9.7 Build and release pipeline.....	32
10. Infrastructure Sizing.....	33
10.1 Load assumptions.....	33
10.2 On-premise servers.....	33
10.3 Cloud services.....	34
10.4 Database sizing.....	34
10.5 Document storage.....	35
10.6 Backup, RPO and RTO.....	35
10.7 Network ports and firewall rules.....	36
11. Non-functional Design.....	37
11.1 Monitoring and health.....	37
11.2 Logging.....	37
12. Technology Stack and Versions.....	39
13. Architecture Decisions.....	41
Appendix A: API Operations.....	42
Appendix B: Glossary.....	46

1. Purpose and Scope

1.1 Purpose

This document describes the solution architecture of SalesVerse 2.0 as configured for Insurans Islam Family Takaful Sendirian Berhad (IIFT) as the Agent/Banca Portal and Back-office. It explains how the solution is structured, how it handles data, integration and security, how it is deployed and sized, and why the main design decisions were taken. It is the Solution Architecture deliverable (DEL-06) and the reference for the Technical Design (DEL-07), the Interface Specification (DEL-09) and the deployment plan (DEL-20).

1.2 Scope

In scope	Out of scope
Agent/Banca Portal for agents, main agents and bank officers Back-office for IIFT staff Participant e-signature page REST API, database, document store, scheduled jobs Integration with Core, FIN, AML provider, SMTP, SMS and Active Directory Deployment, sizing, security, monitoring and DR design	Changes inside IITH core and financial systems Claims assessment and payment (core system) Infrastructure procurement and data-centre operation (IIFT/IITH) Commercial terms (see the proposal)

1.3 Audience

IIFT and IITH IT architecture, infrastructure, security and operations teams; the IIFT project team; iorta TechNXT engineering and support. Readers are assumed to know web applications and relational databases.

1.4 Status of the solution

SalesVerse 2.0 is a working application. The table separates what runs in the current build from what is completed during implementation, so that every statement in this document can be checked against the source code.

Capability	Today	Implementation work
Agent/Banca Portal and Back-office web application (React SPA)	Running	–
REST API with all functional modules listed in chapter 4	Running	–
PostgreSQL schema, migrations, reference data and demonstration data	Running	–
Sessions, CSRF, Argon2id, lockout, RBAC, data scoping, maker-checker	Running	–
Field-level and document encryption, append-only audit trigger	Running	–
Transactional outbox, retry, dead-letter, integration monitor, reconciliation	Running	–
Core, FIN, AML, SMS, SMTP and LDAP adapters	Running in simulated mode	Endpoint mapping and payload formats agreed in the interface specification (DEL-09)
Container images, nginx web tier, Docker Compose, CI pipeline	Running	CD stages to SIT, UAT and PROD
Health endpoints and Prometheus metrics	Running	Dashboards, alert rules and log shipping
Multi-factor authentication for back-office users	–	Delivered during implementation
PostgreSQL streaming replication, pgBackRest backups, DR site	–	Delivered during implementation

Capability	Today	Implementation work
Data retention and archival job; key rotation tooling	–	Delivered during implementation
S3-compatible document storage adapter (only if object storage is chosen)	–	Delivered during implementation
IIFT actuarial rates, wakalah and tabarru' parameters	Indicative values loaded	IIFT values loaded and verified in design and SIT
Legacy data migration	–	Delivered during implementation (DEL-19)

Table 1: Solution status at proposal submission

1.5 Sources

- Source code: apps/api (NestJS API, Prisma schema and migrations), apps/web (React SPA), deploy (Docker Compose), .github/workflows (CI).
- docs/api/openapi.json: the API description produced from the code.
- IIFT Request for Proposal, including Appendix 1 (users), Appendix 2 (policy volumes) and Appendix 3 (product flows).
- Companion documents: Data Dictionary and Production Support Handover.

2. Architecture Principles

These principles guided every design decision recorded in chapter 13.

Principle	Rationale	Implication
One system of record	Every business fact lives in PostgreSQL; external systems receive copies through the outbox.	No dual writes; integrations are replayable; reconciliation compares against one source.
Modular monolith	26 named users and about 600 policies a year do not justify microservices.	One deployable unit; modules follow RFP areas and talk through services, so a module can be split out later (NFR-20).
Server-side enforcement	The browser is untrusted.	Authentication, permissions, data scope, maker-checker and every business rule are enforced by the API; the SPA only hides what a user cannot do.
Atomic business transactions	NFR-16 requires no partial updates.	A business change, its audit row and its outbox message commit in one database transaction.
Configuration over code	Rates, parameters, workflows and master data change more often than code.	Products, parameters, roles, workflows and code lists are maintained in the back-office and audited.
Secure by default	Regulated personal and financial data.	Deny by default; encrypted identifiers; production start-up refuses insecure settings; generic errors with correlation id.
Operable	IIFT and IITH must be able to run it.	Standard containers, JSON logs, health and metrics endpoints, documented runbooks, open-source stack without licence fees.
Testable	Changes must be safe to release.	Unit and end-to-end tests against a real PostgreSQL database run in CI on every change.

Table 2: Architecture principles

3. System Context

SalesVerse 2.0 sits between IIFT's distribution partners and the IITH group systems. Agents and bank officers work in the portal; IIFT staff work in the back-office on the internal network; participants only see a one-time e-signature page. The core and financial systems stay the systems of record for in-force business and accounting, and receive every relevant event from the portal.

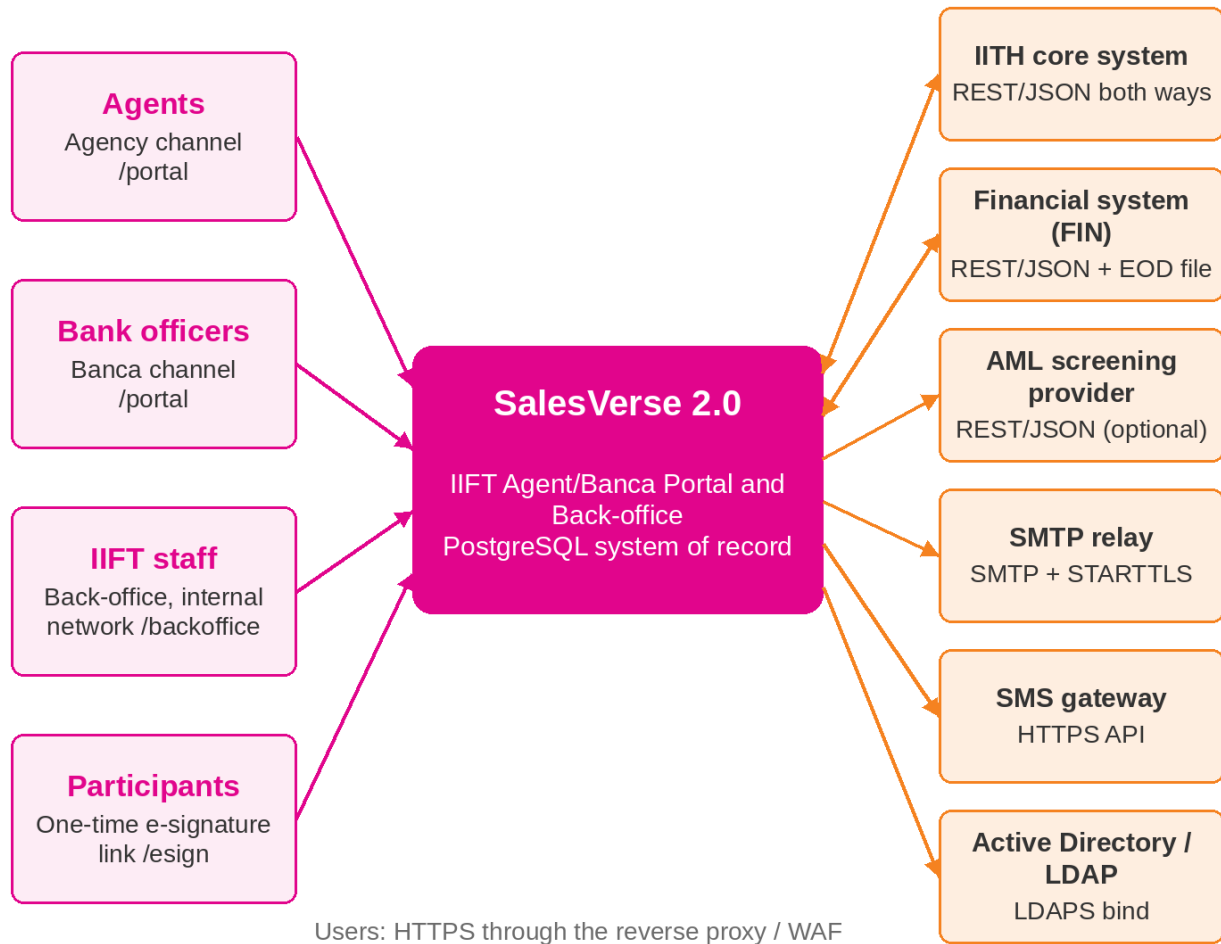


Figure 1: System context

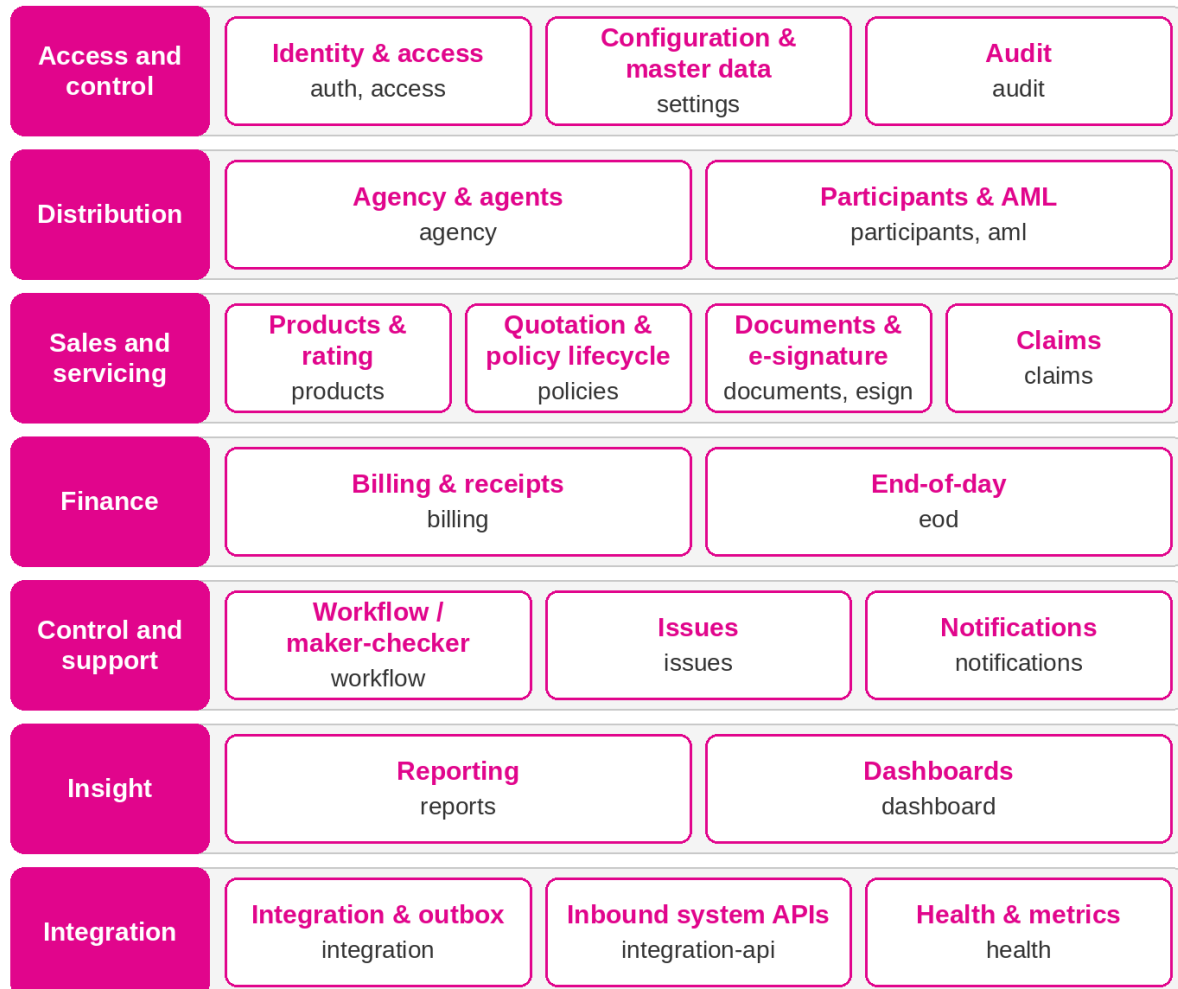
Actor or system	Type	Channel	Interaction	RFP
Agents and main agents	Person	Agent/Banca Portal over HTTPS	Quotations, participants, policies, payments, claims, issues, reports	AP-01 to AP-62
Bank officers and supervisors	Person	Agent/Banca Portal over HTTPS	Financing takaful sales at bank branches; bank-wide view for supervisors	AP-09, AP-59
IIFT staff	Person	Back-office over HTTPS on the internal network	Approvals, administration, compliance, finance, support, reporting	BO-01 to BO-33
Participants	Person	One-time e-signature page, e-mail	Review and sign the application; receive documents	AP-45, AP-62
IITH core takaful system	System	REST/JSON both ways	Agent, participant, policy, endorsement, cancellation and claim events out; agent status and issued-policy queries in	INT-01 to INT-03, INT-09
Financial system (FIN)	System	REST/JSON and EOD file	Receipts, refunds, commission accruals and EOD postings out; posted receipts and commission payments in	INT-04, INT-05, INT-15
AML screening provider	System (optional)	REST/JSON	Name and identifier screening in addition to Compliance watch-lists	AP-16, BO-13

Actor or system	Type	Channel	Interaction	RFP
SMTP relay	System	SMTP with STARTTLS	Notifications, documents, e-signature links, scheduled reports	INT-07
SMS gateway	System	HTTPS API	Short notifications (agency block, approvals)	INT-08
Active Directory / LDAP	System	LDAPS bind	Back-office sign-in for directory accounts	INT-06

Table 3: Actors and external systems

4. Functional Architecture

The API is split into modules that follow the RFP functional areas. Each module owns its controllers, services, data-transfer objects and tests; shared concerns (database access, encryption, numbering, data scope, request context, error handling) live in apps/api/src/common.



Grey text: source folder under apps/api/src/modules.

Figure 2: Functional module map

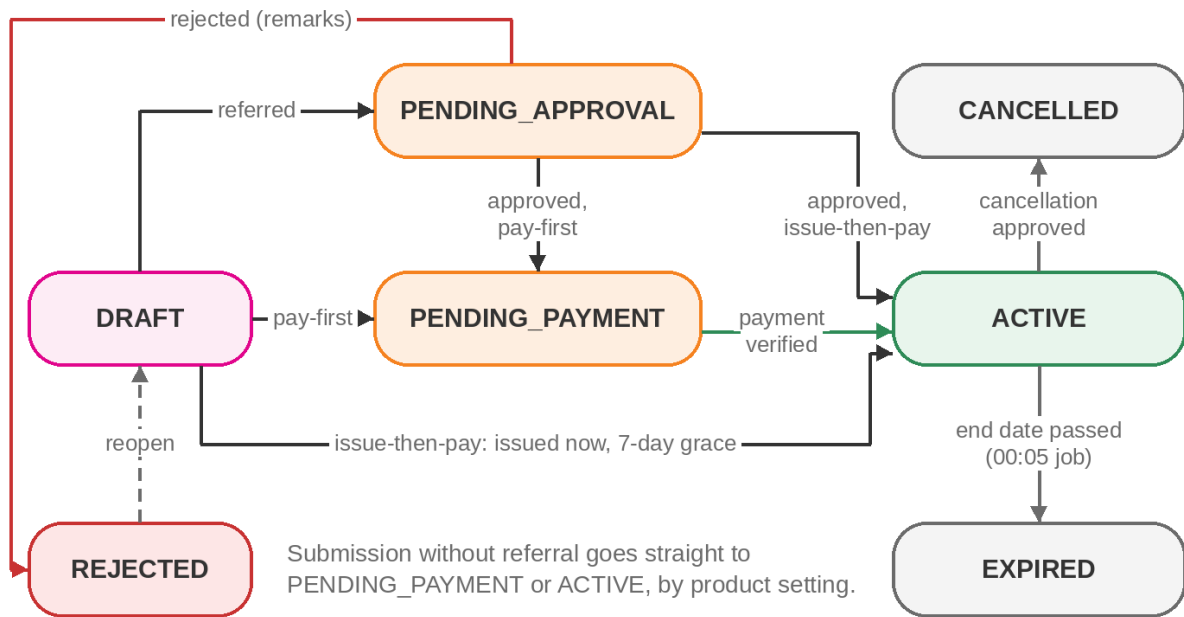
Module	Code	Responsibilities	RFP requirements
Identity & access	auth, access	Sign-in, sessions, password policy and history, lockout, directory sign-in, users, roles, permissions	AP-01 to AP-04, AP-59, BO-01 to BO-04, INT-06, COM-02
Agency & agents	agency	Agencies and banks, agent registration and approval, hierarchy, status, authority limits, issuance block override	AP-05 to AP-10, AP-42, BO-05 to BO-12
Participants & AML	participants, aml	Shared participant master with duplicate check, update requests, watch-list screening, Compliance review	AP-11 to AP-16, AP-48, BO-13 to BO-15
Products & rating	products	Catalogue, FINANCING and FIXED_PLAN rating engines, required documents, questionnaires	AP-17 to AP-19, BO-33
Quotation & policy lifecycle	policies	Quotations, validation, referral, issuance, e-Policy, renewal, endorsement, cancellation, expiry, history	AP-20 to AP-36, AP-44, AP-49 to AP-51, AP-60

Module	Code	Responsibilities	RFP requirements
Billing & receipts	billing	Outstanding contributions, bulk payments with allocation, proof, verification, e-Receipts, commission, grace-period block	AP-30, AP-37 to AP-42, INT-05
Claims	claims	Claim notification with policy validation and documents; back-office status updates	AP-43
Documents & e-signature	documents, esign	Encrypted document store, upload checks, verification queue, PDF output, e-mail of documents, on-screen and link signatures	AP-29, AP-39, AP-44 to AP-47, AP-62, BO-11, BO-12, COM-06
Workflow / maker-checker	workflow	Workflow definitions, approval requests and actions, inbox, handlers	AP-33, AP-49 to AP-51, AP-61, BO-10, BO-16 to BO-19, COM-04
Issues	issues	Issue logging, comments, attachments, assignment, priority, SLA monitoring	AP-55 to AP-57, BO-29 to BO-31
Notifications	notifications	In-app, e-mail and SMS notifications with retry and delivery log	AP-34, AP-54, INT-07, INT-08, COM-05
Reporting	reports	Report catalogue, filters, preview, export (XLSX, CSV, PDF), schedules	AP-58, BO-22 to BO-25, COM-08
Dashboards	dashboard	Agent dashboard, pending actions, management KPIs	AP-52, AP-53, BO-20, BO-21
Audit	audit	Append-only audit trail, search and facets	AP-35, BO-26 to BO-28, COM-03, NFR-11
Configuration & master data	settings	Business parameters with ranges, code lists	BO-32, BO-33, COM-09
Integration & outbox	integration, integration-api	Outbox, dispatcher, gateways, integration log, monitor and retry, reconciliation, inbound system APIs	INT-01 to INT-05, INT-09 to INT-15
End-of-day	eod	EOD issuance report, FIN interface file, EOD posting, reconciliation, re-run	Appendix 3 (FFR01-08 to FFR05-08), INT-04, INT-15
Health & metrics	health	Liveness, readiness, Prometheus metrics	NFR-26, NFR-27

Table 4: Modules and the requirements they cover

4.1 Policy lifecycle

A quotation and the policy it becomes are one record, so history, documents and payments stay attached from first quote to expiry. On submission the API re-rates the quotation with current rates, checks validity (30 days by default), the agency issuance block, required documents, nominees, participant signature and AML status, then decides whether the application must be referred.



Payment status runs alongside: UNPAID, PENDING_VERIFICATION, PAID.

An ACTIVE policy UNPAID after payment_due_date blocks new business for its agency.

Figure 3: Quotation and policy lifecycle

Referral reason	Source
Sum covered above the agent's authority limit	agent.authority_limit (AP-60)
Product limit, e.g. financing amount above B\$150,000 (high-risk limit)	Product configuration
Questionnaire answer marked 'refer if yes'	product.questionnaire
Quality check before issuance for products that require it	Product rules

Table 5: Referral triggers

Pay-first products (the annual plans) wait in PENDING_PAYMENT until Finance verifies payment, then issue. Issue-then-pay products (the financing plans) issue immediately with a payment due date of issuance plus the grace period (7 days by default). A daily job blocks every agent of an agency from submitting new business while any of its issued policies is unpaid beyond the due date, and lifts the block automatically once nothing is overdue (AP-42). An administrator can lift a block manually with a reason; the next daily run re-applies it if contributions are still overdue.

4.2 Maker-checker workflows

Eight transaction types run through the workflow engine. Each type has a configurable sequence of levels; each level names the permission a checker needs and, optionally, an amount threshold from which it applies. The levels are resolved when the request is raised and stored with it, so a later configuration change does not alter requests already in flight. When no level applies the request is approved immediately and the change is applied in the same transaction.

Type	Default levels (seed data)	Effect on final approval
AGENT_REGISTRATION	Operations supervisor approval (bo.approve.agents)	Agent ACTIVE, portal user and role created, AGENT_UPSERT to Core
AGENT_PROFILE_UPDATE	Operations supervisor approval	Changes applied, AGENT_UPSERT to Core
AGENT_STATUS_CHANGE	Operations supervisor approval	Status applied; sessions revoked when deactivated
PARTICIPANT_UPDATE	Operations supervisor approval (bo.approve.participants)	Changes applied; re-screened on name change; PARTICIPANT_UPSERT to Core

Type	Default levels (seed data)	Effect on final approval
POLICY_REFERRAL	Underwriting / quality check; second approval from sum covered B\$300,000	Policy proceeds to PENDING_PAYMENT or ACTIVE; rejection returns it with remarks
POLICY_ENDORSEMENT	Policy servicing approval (bo.approve.servicing)	Endorsement applied, POLICY_ENDORSED
POLICY_CANCELLATION	Policy servicing approval	CANCELLED, POLICY_CANCELLED, REFUND_REQUESTED when a refund is due
PAYMENT_VERIFICATION	Finance verification (bo.approve.payments)	Receipts issued, RECEIPT_POSTED, pay-first policies issued

Table 6: Approval types

Rules enforced by the server: the maker can never approve their own request; the same person cannot approve two levels of one request; rejection requires remarks; a request can be withdrawn by its maker until decided; only one pending request per record and type is allowed; and a decision fails with STALE_RECORD if another checker decided the same level first.

4.3 Scheduled processing

Job	Schedule (Brunei time)	Lock	Purpose	Source
end-of-day	Daily 23:30 (EOD_CRON, JOBS_TIMEZONE)	end-of-day	EOD report, FIN file, EOD_POSTING message, receipt reconciliation	eod/eod.service.ts
policy-expiry	Daily 00:05	policy-expiry	ACTIVE policies past end date become EXPIRED; agent notified	policies/policy.jobs.ts
payment-grace-period	Daily 01:00	payment-grace-period	Blocks agencies with overdue issue-then-pay policies; lifts the block when settled	billing/grace-period.service.ts
outbox-dispatch	Every 20 seconds	outbox-dispatch	Delivers up to 25 due outbox messages to Core and FIN	integration/outbox.dispatcher.ts
notification-dispatch	Every 30 seconds	notification-dispatch	Sends up to 50 pending e-mail and SMS notifications	notifications/notification.dispatcher.ts
issue-sla-monitor	Every 5 minutes	issue-sla-monitor	Flags issues past their response or resolution target and alerts support	issues/issues.service.ts
scheduled-reports	Every 15 minutes	scheduled-reports	Runs due report schedules and e-mails the files	reports/reports.service.ts
session pruning	Every 15 minutes	none (idempotent)	Deletes expired rows from user_session	connect-pg-simple

Table 7: Scheduled jobs

Each job takes a PostgreSQL transaction-scoped advisory lock (pg_try_advisory_xact_lock) named after the job before it runs. Every API replica schedules the jobs, but only the replica that gets the lock does the work; the lock is released automatically if the process stops. JOBS_ENABLED=false disables all jobs in an instance, for example in a reporting replica.

5. Logical and Application Architecture

5.1 Layers

The solution has five layers. Two single-page applications, built from one React code base, run in the browser. An nginx web tier serves them and forwards /api calls. The NestJS API holds all business logic and security enforcement. PostgreSQL holds all structured data, and an encrypted file store holds documents. Scheduled jobs run inside the API processes and coordinate through database locks, so there is no separate scheduler or message broker to operate.

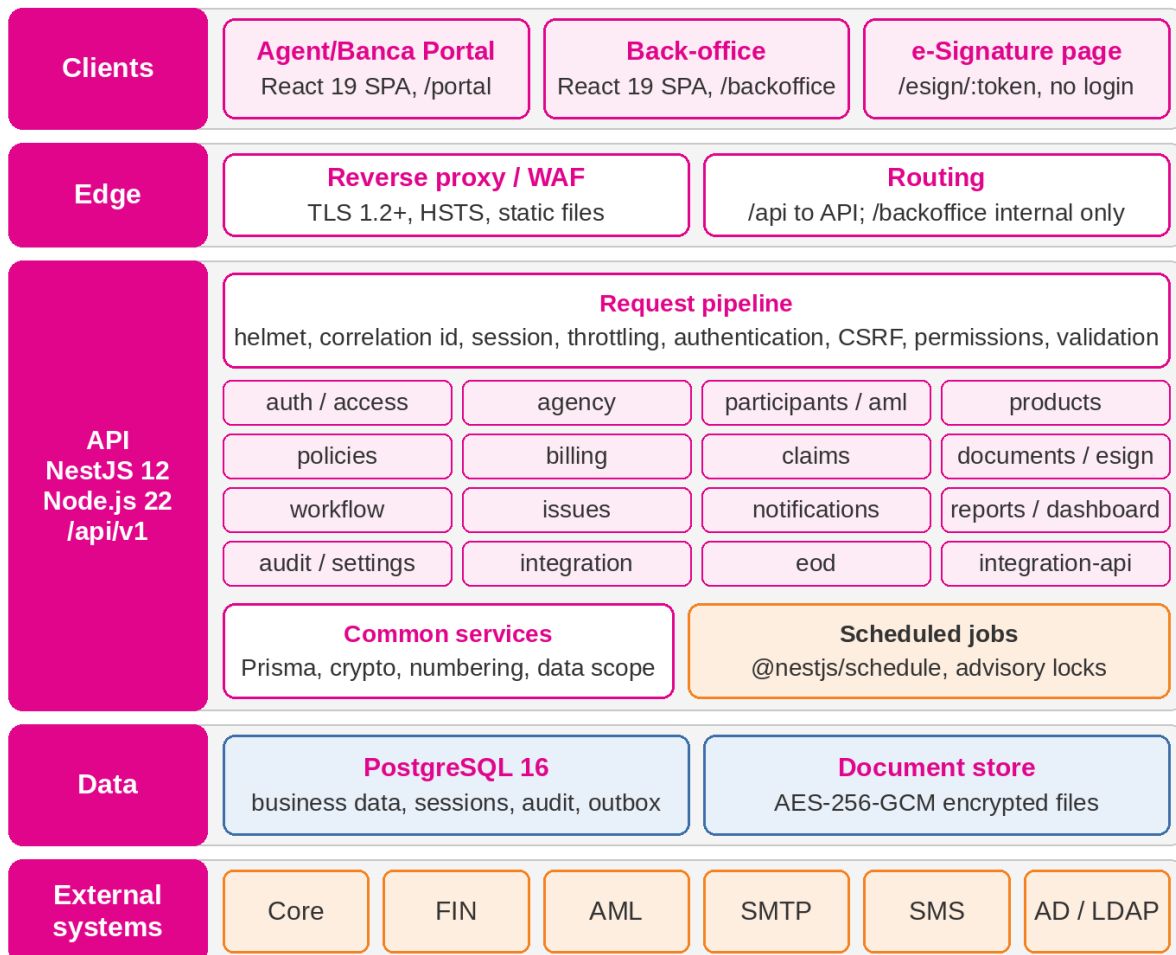


Figure 4: Logical architecture

5.2 Front end

apps/web is a React 19 and TypeScript application built with Vite 7 and Ant Design 6, themed in the Insurans Islam TAIB colours. It contains 54 page components behind three route trees: /portal for agents and bank officers, /backoffice for IIFT staff and /esign/:token for participants. Pages are loaded on demand, and vendor libraries are split into separate chunks (react, antd, charts) so the first screen stays small.

Server state is managed with TanStack Query. A thin API client sends the session cookie (same origin), adds the X-CSRF-Token header to every state-changing call, and turns error responses into a typed error that carries the correlation id the user can quote to support. A 401 response returns the user to the sign-in page. Route guards in the SPA only shape navigation; every check is repeated by the API.

5.3 API

The API is a NestJS 12 application on Node.js 22, written in TypeScript as ES modules. All business endpoints are under `/api/v1` and are described in `docs/api/openapi.json` (128 paths, 149 operations). Namespaces separate the audiences, so the reverse proxy can publish the back-office only on the internal network.

Namespace	Ops	Purpose	Access control
<code>/api/v1/auth</code>	4	Sign-in, sign-out, current user, password change	Session (login is public)
<code>/api/v1/portal</code>	45	Agent/Banca Portal functions	Session, PORTAL audience, permissions, data scope
<code>/api/v1/backoffice</code>	80	Back-office functions	Session, BACKOFFICE audience, permissions
<code>/api/v1/common</code>	15	Functions used by both audiences: documents, notifications, issues, products, codes	Session, permissions, data scope
<code>/api/v1/public</code>	2	Participant e-signature page	One-time token (hash stored)
<code>/api/v1/integration</code>	3	System-to-system APIs for Core and FIN	API key (SHA-256 held), IP allow-list
<code>/health/live</code> , <code>/health/ready</code> , <code>/metrics</code>	3	Probes and Prometheus metrics (outside <code>/api/v1</code>)	Public probes; METRICS_TOKEN for metrics

Table 8: API namespaces

5.4 Request processing

Every request passes the same pipeline, configured once in `app.setup.ts` and used by both the server and the end-to-end tests. Global guards run in a fixed order: throttling, authentication, CSRF, then permissions. A route is protected unless it is explicitly marked public.

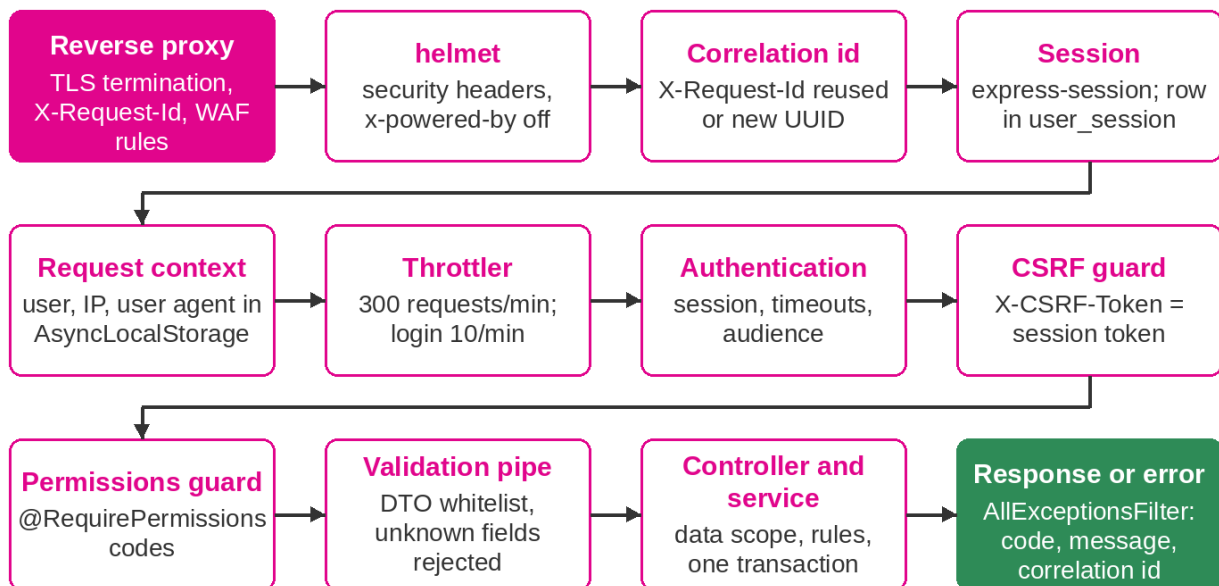


Figure 5: Request pipeline

5.5 Error handling

A global exception filter turns every error into the same JSON body: `statusCode`, a stable code, a message safe to show, optional details and the correlation id. Unexpected errors are logged in full on the server and reach the user only as a generic message with the correlation id (NFR-15).

HTTP	Codes	When
400	BAD_REQUEST	Validation failed; details lists each field problem

HTTP	Codes	When
401	NOT_AUTHENTICATED, INVALID_CREDENTIALS, ACCOUNT_LOCKED, SESSION_EXPIRED	No valid session or sign-in refused
403	FORBIDDEN, CSRF_TOKEN_INVALID, WRONG_AUDIENCE, PASSWORD_CHANGE_REQUIRED	Not permitted
404	NOT_FOUND	Record does not exist or is outside the user's data scope
409	DUPLICATE, STALE_RECORD	Unique value exists; record changed by another user
413	PAYLOAD_TOO_LARGE	Body or file above the limit
422	Business rule codes, e.g. AGENCY_BLOCKED, SUBMISSION_INCOMPLETE, QUOTATION_EXPIRED	Valid request that breaks a business rule
429	TOO_MANY_REQUESTS	Rate limit reached
500	INTERNAL_ERROR	Unexpected error; logged with stack trace on the server only
503	NOT_READY, NOT_CONFIGURED	Database unavailable; inbound integration not configured

Table 9: Error responses

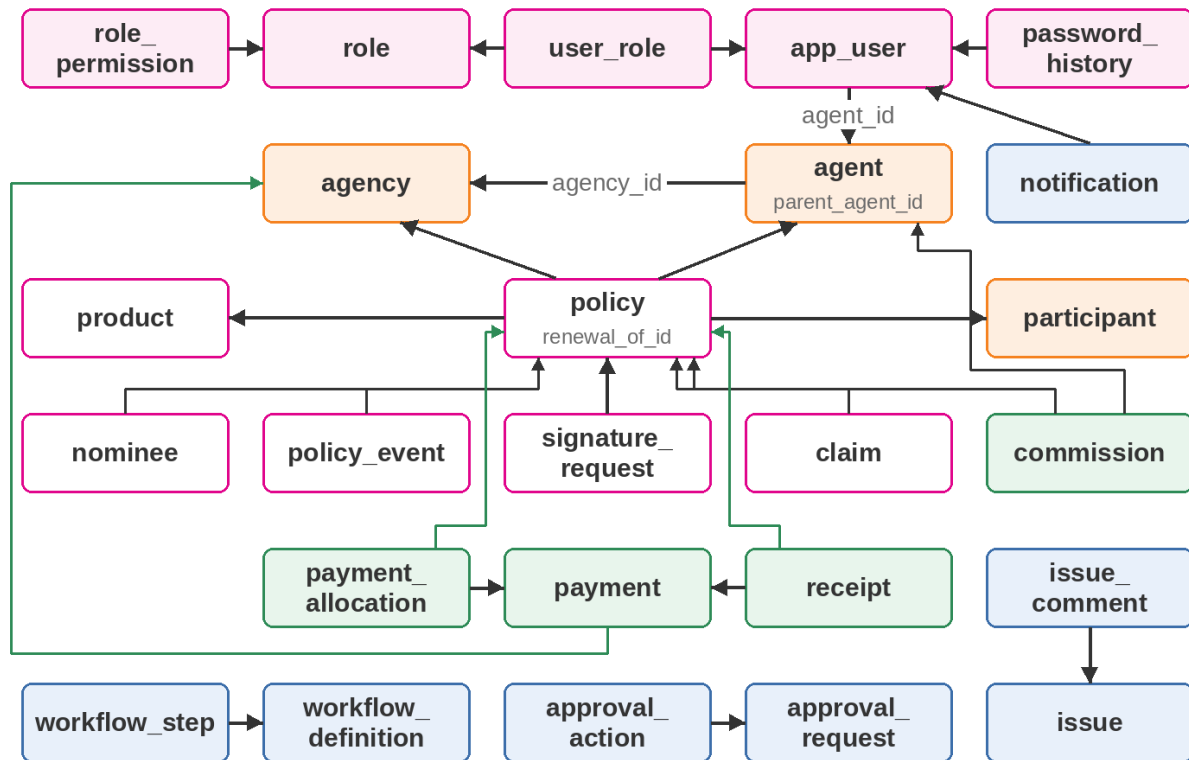
5.6 Consistency and concurrency

- Each business operation runs in one database transaction that includes its audit row and outbox message.
- Records edited by several users (agent, participant, policy) carry a version column; updates quote the version read and fail with STALE_RECORD if it changed.
- Approval decisions update the request only if it is still PENDING at the expected level, so two checkers cannot both decide.
- Reference numbers come from PostgreSQL sequences, which are safe across replicas without row locks.
- Amounts are DECIMAL end to end and rounded half-up to two places; binary floating point is never stored.

6. Data Architecture

The database has 37 tables in 13 areas. The Data Dictionary documents every column; this chapter explains the structure and the controls around it.

6.1 Entity overview



Tables without foreign keys (polymorphic owner, logs or configuration)

document, audit_log, outbox_message, integration_log, reconciliation_run, eod_run, report_schedule, config_parameter, code_item, aml_screening, aml_watchlist_entry, user_session

Arrow: child table points to its parent (foreign key). Grey text: self-reference.

Figure 6: Entity overview

Entity	Role in the model
agency, agent	Distribution structure. Agents belong to one agency or bank; sub-agents point to their main agent. Agency carries the issuance block.
app_user, role, user_role, role_permission	Sign-in identities and role-based permissions. A portal user is linked to exactly one agent.
participant	One shared record per identification number across agencies; encrypted identifier with blind index; AML status.
product	Rating configuration, required documents and questionnaire as validated JSON.
policy	Quotation and policy in one row through the whole lifecycle; contribution breakdown and risk details as JSON; outstanding amount and due date.
payment, payment_allocation, receipt, commission	Bulk payments allocated per policy; one receipt per policy; commission accrued at issuance.
approval_request, approval_action	Maker-checker requests with payload and resolved levels; full action history.

Entity	Role in the model
document	Metadata for encrypted files attached to any owner type.
audit_log	Append-only audit trail with before and after values.
outbox_message, integration_log, reconciliation_run, eod_run	Integration delivery, call log, reconciliation and end-of-day history.

Table 10: Key entities

6.2 Why PostgreSQL

PostgreSQL 16 was chosen over MongoDB because IIFT's data is relational and financial, and because issuance, receipting, audit and integration must commit together.

Criterion	PostgreSQL 16 (selected)	MongoDB
Data shape	Agency, agent, sub-agent; policy, participant, nominees, payments, receipts, claims: strongly relational	Suits self-contained documents with few cross-references
Transaction integrity (NFR-16)	Issuance, receipt, audit and outbox commit in one ACID transaction	Multi-document transactions exist but are not the natural model
Financial accuracy	Exact DECIMAL, foreign keys, CHECK and UNIQUE constraints in the database	Decimal128; most integrity rules move into application code
Reporting (BO-22 to BO-25)	Joins and aggregates in SQL	Aggregation pipelines with \$lookup
Flexible structures	JSONB for product configuration, questionnaires, audit values, outbox payloads	Native
Operations	Streaming replication, PITR, pgBackRest; managed service on AWS and Azure; skills common in IITH	Replica sets; a second database skill set
Licence	PostgreSQL licence, no fee	SSPL; some enterprise features paid
Fit to volume	Under 1 GB of rows after five years at Appendix 2 volumes	Strength is very large schemaless data, not needed here

Table 11: Database selection

6.3 Integrity controls

- 27 foreign keys with explicit ON DELETE rules: RESTRICT for business parents, CASCADE only for dependent detail rows (nominees, events, allocations, comments, role links).
- 24 unique indexes, including one participant and one agent per identification type and number, one receipt number, one policy number.
- 4 CHECK constraints: amounts on policy, payment and allocation; nominee share between 0 and 100.
- Validation of every request body against a whitelist DTO before it reaches a service; unknown fields are rejected (NFR-30).
- JSON columns (product configuration, questionnaire, payloads) are validated by the owning service before they are written.

6.4 Audit trail

The audit service writes one row per security or business event: actor, action, entity, before and after values, IP address, user agent and correlation id. It writes inside the caller's transaction, so an audit row exists exactly when the change exists. Passwords, hashes, encrypted identifiers, tokens and storage keys are replaced with a fixed REDACTED marker. In the database, the trigger `audit_log_no_update` calls `audit_log_block_mutation()` before any UPDATE or DELETE and raises an exception, so the trail is append-only whichever client connects. Granting the application role only INSERT and SELECT on `audit_log`, with a separate migration owner role, is set up during implementation.

6.5 Field-level encryption

IC, passport and business registration numbers of agents and participants, and nominee identification numbers, are encrypted by the application before they reach the database: AES-256-GCM with a random 96-bit IV per value, stored as v1:<base64(IV | tag | ciphertext)>. The version prefix allows a new key to be introduced without a big-bang re-encryption. For exact search and duplicate prevention, the application also stores a blind index: HMAC-SHA256 of the normalised number (separators removed, upper case) with a separate key. The unique constraint on (id_type, id_number_hash) therefore blocks duplicates without decrypting anything. Screens show the number masked to its last four characters.

6.6 Reference numbers

Sequence	Format	Column
seq_agent_code	AG-000045 (agency) or BK-000012 (bank officer)	agent.agent_code
seq_participant_no	PT/26/000123	participant.participant_no
seq_quotation_no	QT/26/000123	policy.quotation_no
seq_policy_no	PRO/26/000045 (product code prefix)	policy.policy_no
seq_payment_no	PY/26/000123	payment.payment_no
seq_receipt_no	RC/26/000123	receipt.receipt_no
seq_claim_no	CL/26/000123	claim.claim_no
seq_issue_no	IS/26/000123	issue.issue_no
seq_request_no	RQ/26/000123	approval_request.request_no

Table 12: Business reference numbers

The two-digit year is the calendar year of issue. Sequences never repeat a value; a rolled-back transaction can leave a gap, which is acceptable for these references and is explained to auditors.

6.7 Documents

Uploaded and generated files are encrypted with AES-256-GCM (separate key, one IV per file) and written to the document store under a generated key of the form yyyy/mm/<uuid>.bin, so no user input ever influences a file path. The document table holds the owner, type, sanitised file name, detected content type, size, SHA-256 checksum and verification status. The store is a directory: a local or NFS volume on-premise, or an encrypted Amazon EFS or Azure Files share in the cloud. An S3-compatible adapter is delivered during implementation if IIFT prefers object storage.

6.8 Configuration data

Business parameters live in config_parameter and are changed in the back-office within the allowed range; every change is audited. Values are cached briefly in each API instance.

Parameter	Default	Range	Meaning
security.password.min_length	12	8 to 64	Minimum password length
security.password.require_upper	true	–	Password must contain an upper-case letter
security.password.require_lower	true	–	Password must contain a lower-case letter
security.password.require_digit	true	–	Password must contain a digit
security.password.require_symbol	true	–	Password must contain a symbol
security.password.history_count	5	0 to 24	Number of previous passwords that cannot be reused
security.password.expiry_days	90	0 to 365	Days before a password must be changed (0 = never)
security.lockout.max_attempts	5	3 to 20	Failed login attempts before the account is locked
security.lockout.minutes	30	1 to 1440	Lock duration in minutes (administrator can unlock earlier)
security.session.idle_minutes	15	5 to 120	Session idle timeout in minutes
security.session.absolute_hours	8	1 to 24	Maximum session lifetime in hours

Parameter	Default	Range	Meaning
security.session.single_active	true	–	Signing in ends the user's other sessions
billing.payment_grace_days	7	1 to 60	Days after issuance within which contribution must be submitted before the agency is blocked
policy.quotation_validity_days	30	1 to 180	Days a draft quotation remains valid
policy.renewal_notice_days	45	7 to 120	Days before expiry that a policy is listed as due for renewal
policy.require_participant_signature	true	–	Participant signature is mandatory before submission
policy.esign_link_hours	72	1 to 336	Validity of e-signature links sent by email (hours)
aml.match_threshold	85	50 to 100	Name-match score (0-100) at or above which a screening is flagged for review
commission.default_rate	0.10	0 to 1	Default commission / referral fee rate when the product does not define one
sla.critical.response_hours	1	1 to 72	Critical issue response target (hours)
sla.critical.resolution_hours	4	1 to 168	Critical issue resolution target (hours)
sla.high.response_hours	2	1 to 72	High issue response target (hours)
sla.high.resolution_hours	24	1 to 336	High issue resolution target (hours)
sla.medium.response_hours	8	1 to 168	Medium issue response target (hours)
sla.medium.resolution_hours	72	1 to 720	Medium issue resolution target (hours)
sla.low.response_hours	24	1 to 336	Low issue response target (hours)
sla.low.resolution_hours	168	1 to 2160	Low issue resolution target (hours)
integration.max_attempts	6	1 to 20	Delivery attempts before an integration message is moved to dead-letter
notification.max_attempts	5	1 to 20	Email/SMS delivery attempts before giving up
document.expiry_warning_days	30	1 to 180	Days before expiry that documents are flagged

Table 13: Business parameters and defaults

6.9 Retention and archival

Proposed retention periods are below. They are confirmed with IIFT Compliance against the PDPO 2025, AMBD requirements and AML/CFT record-keeping rules during design. The retention job that applies them, with an archive schema and audit of every purge, is delivered during implementation. Expired sessions are already pruned automatically.

Data	Retention	Mechanism
Policies, quotations taken up, nominees, policy events	7 years after policy end or cancellation	Archive to read-only schema, then delete
Quotations not taken up (DRAFT, REJECTED)	2 years after last change	Delete with documents
Participants and AML screenings	7 years after the last policy ends (AML/CFT record keeping)	Archive, then delete
Payments, receipts, commissions, EOD runs, FIN files	7 years after the financial year	Archive, then delete
Audit log	7 years online or in archive	Controlled archive procedure (trigger blocks normal deletion)
Integration log, delivered outbox messages	2 years	Delete
Notifications	1 year (sensitive bodies already removed after delivery)	Delete
Issues and comments	3 years after closure	Delete
Sessions	Until expiry	Pruned every 15 minutes (running today)

Table 14: Proposed retention

6.10 Data migration

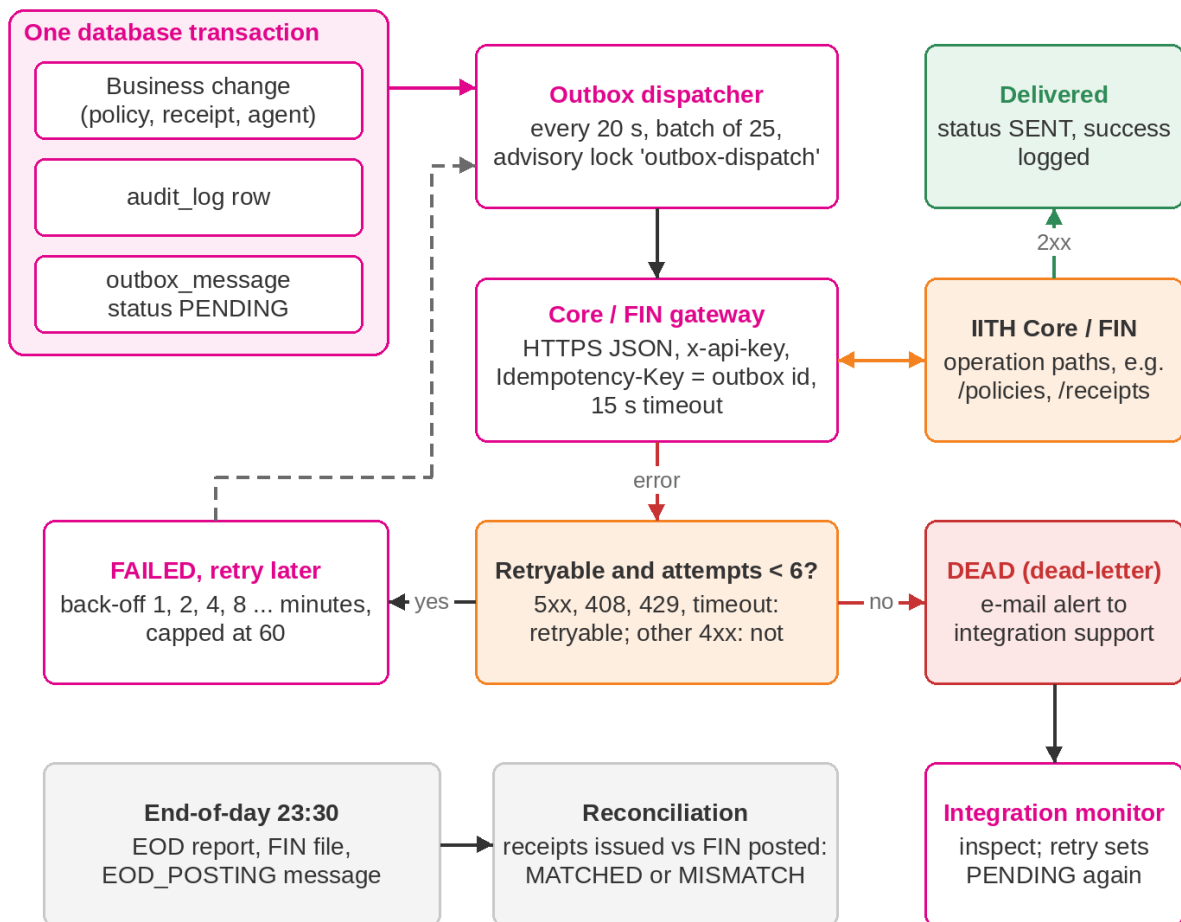
Legacy agents, agencies, participants and in-force policies are loaded through the same services the application uses, so encryption, blind indexes, numbering and audit apply to migrated records. Each

rehearsal produces a reconciliation of counts and amounts that IIFT signs off (DEL-19); the migration tooling is delivered during implementation.

7. Integration Architecture

7.1 Approach

Outbound business events to the core and financial systems use a transactional outbox. Calls that need an answer at once (AML screening, directory sign-in, FIN posted receipts) are synchronous with timeouts. E-mail and SMS go through the notification table and its own dispatcher. Core and FIN call the portal through a small set of API-key-protected endpoints. Every call in either direction is written to `integration_log`, which feeds the integration monitor (INT-11, INT-14).



Inbound: Core and FIN call `/api/v1/integration/*` with an API key; every call is written to `integration_log`.

Figure 7: Outbox delivery, retry and dead-letter

7.2 Delivery guarantees

- **Atomic capture.** The outbox row is inserted in the same transaction as the business change, so a message is never lost and never sent for a change that rolled back.
- **At-least-once delivery with idempotency.** The outbox id is sent as the Idempotency-Key header; the receiver must ignore a key it has already processed. This is part of the interface specification.
- **Retry with back-off.** Connection errors, timeouts, HTTP 5xx, 408 and 429 are retried after 1, 2, 4, 8 ... minutes, capped at 60 minutes. Other 4xx responses mean the message is wrong and go straight to dead-letter.
- **Dead-letter and alert.** After `integration.max_attempts` attempts (6 by default) the message becomes DEAD and every holder of `bo.integration.manage` receives an e-mail. The integration monitor shows the message and error; Retry resets it to PENDING.

- **Ordering.** Due messages are taken oldest first in batches of 25 by a single dispatcher (advisory lock), so events for one record are delivered in order unless one of them is retrying.
- **Timeouts.** Each endpoint has its own timeout (15 seconds by default).

7.3 Outbound operations

System	Operation	Endpoint (base URL from configuration)	Raised when
CORE	AGENT_UPSERT	POST /agents	Agent registration, profile update or status change approved
CORE	PARTICIPANT_UPSERT	POST /participants	Participant update approved
CORE	POLICY_ISSUED	POST /policies	Policy issued: directly, after referral approval or after payment verification
CORE	POLICY_ENDORSED	POST /policies/endorsements	Endorsement approved
CORE	POLICY_CANCELLED	POST /policies/cancellations	Cancellation approved
CORE	CLAIM_NOTIFIED	POST /claims	Claim notification submitted
FINANCE	RECEIPT_POSTED	POST /receipts	Payment verified and receipt issued
FINANCE	REFUND_REQUESTED	POST /refunds	Cancellation approved with a refund due
FINANCE	COMMISSION_ACCRUED	POST /commissions	Commission accrued at issuance
FINANCE	EOD_POSTING	POST /eod-postings	End-of-day run completed

Table 15: Outbox operations

System	Call	Used by	Content
FIN	GET /receipts/posted? date=	Reconciliation in live mode	Receipt references posted for a date
AML provider	POST /screen	Agent or participant screening (when AML_API_BASE_URL is set)	Name, identification number, date of birth, nationality; returns matches with scores
SMS gateway	POST /messages	Notification dispatcher	Sender id, mobile number, text up to 480 characters
SMTP relay	SMTP, STARTTLS required	Notification dispatcher	Message with optional attachments
Active Directory	LDAP bind and search	Back-office sign-in of DIRECTORY users	Service bind, user search with escaped filter, user bind

Table 16: Synchronous and notification calls

Endpoint paths, payload fields and the FIN interface file layout shown here are the working defaults. They are confirmed or mapped to IITH's formats in the Interface Specification (DEL-09) during design.

7.4 Inbound system APIs

Endpoint	Caller	RFP	Input	Effect
PUT /api/v1/integration/agents/{agentCode}/status	Core	INT-02	{status, reason}; status ACTIVE, INACTIVE, SUSPENDED or TERMINATED	Updates the agent and its user; ends all sessions of a deactivated agent
GET /api/v1/integration/policies? issuedOn=YYYY-MM-DD	Core, FIN	INT-09	Business date	Policies issued that day with product, participant, agent, agency, amounts and dates
POST /api/v1/integration/commissions/paid	FIN	INT-05	items: list of {policyNo, agentCode, paidOn}, up to 1,000 items	Marks commissions PAID; returns updated count and unmatched items

Table 17: Inbound endpoints

Callers present x-api-key. The API holds only the SHA-256 of the key (INBOUND_API_KEY_SHA256) and compares in constant time; if no hash is configured the endpoints answer 503. nginx or the firewall restricts /api/v1/integration to the Core and FIN host addresses. Each call is logged as INBOUND with its outcome and duration.

7.5 End-of-day and reconciliation

At 23:30 Brunei time (EOD_CRON) the end-of-day job selects policies issued and receipts issued during the business day and, in one transaction, stores the EOD issuance report (Excel: summary by product, policies issued, receipts), stores the FIN interface file (CSV: one PI line per policy and one RC line per receipt with amounts in BND), queues an EOD_POSTING message with the day's totals and records the run. It then reconciles receipts: in live mode against the receipt references FIN reports as posted for that date; in simulated mode against RECEIPT_POSTED messages delivered from the outbox. The result is MATCHED or MISMATCH with the unmatched receipt numbers. Finance can re-run EOD for any past date from the back-office; the eod_run row for the date is updated, not duplicated. A re-run queues a fresh EOD_POSTING carrying the same business date, so the interface specification defines the business date as FIN's replacement key for EOD postings.

7.6 Simulated and live modes

INTEGRATION_MODE=simulated records every outbound message and marks its reference as simulated without calling external systems, so the full flow can be built and tested before IITH endpoints exist. In live mode the configured endpoints are called; a production instance in live mode refuses to start without the Core, FIN, SMTP and inbound key settings. E-mail and SMS are sent only when SMTP_HOST and SMS_API_BASE_URL are set, independent of the mode.

8. Security Architecture

Controls are layered so that no single failure exposes data. They are aligned with OWASP ASVS Level 2 and the OWASP Top 10, the Brunei Personal Data Protection Order 2025 and AMBD technology-risk expectations. The Security Assessment Report records the test evidence.

Network	WAF, TLS 1.2+	DMZ / internal split	IP allow-list for system APIs
HTTP	helmet headers, HSTS	rate limiting	2 MB JSON limit
Identity	Argon2id, lockout	server-side sessions	CSRF token, SameSite
Authorisation	audience + permissions	data scope per agency	maker-checker
Data	AES-256-GCM fields	encrypted documents	append-only audit
Operations	redacted JSON logs	secrets outside code	scans and VAPT

Figure 8: Defence in depth

8.1 Control catalogue

Control	Implementation	RFP
Passwords	Argon2id (19 MiB memory, 2 iterations, parallelism 1). Policy: 12+ characters, upper, lower, digit, symbol, must not contain the user name, last 5 not reusable, 90-day expiry, forced change after reset. All values are parameters.	AP-02, NFR-09
Lockout	5 failed attempts lock the account for 30 minutes (parameters); unknown user names take the same time to reject; every success, failure and lock is audited; administrators can unlock.	AP-04
Sessions	Server-side in PostgreSQL; cookie HttpOnly, Secure, SameSite=Strict, path /; id regenerated at sign-in; idle timeout 15 minutes and absolute 8 hours (parameters); single active session option; sign-out and password change revoke sessions on the server.	AP-03
CSRF	Synchroniser token: 32 random bytes created at sign-in, held in the session, returned to the SPA and required in X-CSRF-Token on every state-changing request; compared in constant time.	NFR-14
Authorisation	48 permission codes grouped into configurable roles; portal and back-office permissions cannot be mixed in one role; every route declares its audience and permissions; deny by default.	AP-59, BO-02, NFR-10
Data scope	Back-office: all records. Portal users with agency-wide permission: their agency or bank. Main agents: themselves and sub-agents two levels down. Others: own records. Applied in every portal query.	AP-09, AP-12, COM-02

Control	Implementation	RFP
Maker-checker	Maker cannot approve own request; one person cannot approve two levels; remarks on rejection; levels fixed at submission.	BO-17, COM-04
In transit	TLS 1.2 or later at the load balancer or nginx with HSTS; TLS to PostgreSQL (sslmode=require), STARTTLS to SMTP, LDAPS, HTTPS to all APIs.	NFR-08, COM-10
At rest	AES-256-GCM for identification numbers and documents with keys outside the database; HMAC-SHA256 blind index; encrypted volumes and backups (infrastructure).	COM-01, COM-10, NFR-08
Uploads	Content type from magic bytes (PDF, PNG, JPEG only); MAX_UPLOAD_MB (10 MB) limit; file name sanitised and extension forced to match content; SHA-256 recorded; ClamAV INSTREAM scan, and the upload fails if the scanner cannot be reached.	AP-46, COM-06
Rate limiting	nginx 20 requests/second per IP (burst 40) on /api; API 300 requests/minute per IP; sign-in and password change 10/minute.	NFR-14
Headers	helmet on API responses; nginx adds HSTS, CSP (self only, frame-ancestors none), X-Frame-Options DENY, nosniff, Referrer-Policy and Permissions-Policy; server tokens and x-powered-by removed.	NFR-14
Errors	Generic message plus correlation id to the client; full details only in server logs.	NFR-15
Logging	JSON logs; cookie, authorization, CSRF and set-cookie headers redacted; audit values redacted.	NFR-11, NFR-28
System APIs	API key compared by SHA-256 in constant time; only the hash is configured; IP allow-list at nginx or the firewall; every call logged.	INT-10
Production guard	The API refuses to start in production with a non-secure cookie, API docs enabled, or live integration without Core, FIN, SMTP and inbound key settings.	NFR-14
Multi-factor authentication	E-mail one-time code or authenticator app for back-office users; optional for portal users. Delivered during implementation.	NFR-09

Table 18: Security controls

8.2 Roles

Code	Role	Audience	Purpose
AGENCY_PRINCIPAL	Agency principal / main agent	Portal	Main agent: own business, sub-agents and agency-wide view; registers sub-agents
AGENT	Agent	Portal	Sub-agent: own quotations, policies, payments and claims
BANCA_OFFICER	Bank officer (banca)	Portal	Bank officer selling financing takaful at the branch
BANCA_SUPERVISOR	Bank supervisor (banca)	Portal	Bank branch supervisor: bank-wide view and officer registration
SYSTEM_ADMINISTRATOR	System administrator	Back-office	Users, roles, parameters, master data, workflows, products and integration
OPERATIONS_OFFICER	Operations officer (maker)	Back-office	Agent and agency administration, document checks, claims and issues
OPERATIONS_SUPERVISOR	Operations supervisor (checker)	Back-office	Approves agent, participant and servicing requests
UNDERWRITER	Underwriter / quality check	Back-office	Approves referred quotations and performs quality checks before issuance
FINANCE_OFFICER	Finance officer	Back-office	Verifies payments, issues receipts, runs end-of-day and reconciliation
COMPLIANCE_OFFICER	Compliance officer	Back-office	Reviews AML/KYC cases and maintains watch-lists
MANAGEMENT	Management	Back-office	Management dashboard, KPIs and reports (read-only)
SUPPORT_DESK	Support desk	Back-office	Handles issues reported through the portal
AUDITOR	Internal auditor	Back-office	Read-only access to records and the audit trail

Table 19: Default roles (seed data, editable in the back-office)

8.3 Keys and secrets

All secrets are supplied as environment variables from the platform's secret store (HashiCorp Vault or the IITH equivalent on-premise; AWS Secrets Manager or Azure Key Vault in the cloud) and never stored in the repository or images. Encryption keys are 256-bit random values, base64 encoded; the API checks their length at start-up.

Secret	Purpose	Rotation
SESSION_SECRET	Signs the session cookie	Rotate yearly or on staff change; all users sign in again
FIELD_ENCRYPTION_KEY	Encrypts identification numbers	Version prefix v1 in each value; re-encryption tool for a v2 key delivered during implementation
FIELD_HASH_KEY	Blind index of identification numbers	Rotation needs every blind index recomputed; same tool
DOCUMENT_ENCRYPTION_KEY	Encrypts stored files	Rotation re-encrypts the store; tool delivered during implementation
INBOUND_API_KEY_SHA256	Hash of the key Core and FIN present	Rotate yearly; issue new key, update hash, retire old key
CORE/FINANCE/AML/SMS API keys	Outbound authentication	Per provider policy
SMTP_PASSWORD, LDAP_BIND_PASSWORD	Relay and directory service accounts	Per IITH policy
Database passwords	Application and migration roles	Per IITH policy
METRICS_TOKEN	Protects /metrics	Yearly

Table 20: Secrets

8.4 Security testing

- End-to-end tests cover authentication, access control across audiences and data scopes, and the policy lifecycle against a real database on every CI run.
- tools/security/security-checks.mjs runs repeatable OWASP Top 10 checks against a test environment before each release.
- CI blocks a release on moderate or higher npm audit findings and on unfixed critical or high container image findings.
- An independent VAPT is performed before go-live; no critical or high finding may remain open (NFR-13).

9. Deployment Architecture

9.1 Environments

Env.	Purpose	Hosted by	Shape	Data	Integration	API docs
DEV	Development, sprint demos	iorta	1 VM or laptop compose	Synthetic, demo seed	simulated	Enabled
SIT	Integration and performance testing	IIFT (iorta until ready)	1 VM, compose	Synthetic and masked	live (test endpoints)	Enabled
UAT	Acceptance, training, release rehearsal	IIFT data centre	1 VM, compose	Masked or migrated	live (test endpoints)	Disabled
PROD	Live operation	IIFT/IITH data centre	2 app + 2 DB VMs	Production	live	Disabled
DR	Disaster recovery	IIFT/IITH DR site	1 app + 1 DB VM	Replica of PROD	live (on invocation)	Disabled

Table 21: Environments

Every environment runs the same images; only configuration differs. Promotion is DEV to SIT to UAT to PROD, and nothing reaches PROD that has not passed UAT.

9.2 Option A: on-premise high availability (recommended)

Two application VMs sit behind an external WAF and load balancer pair for internet users and an internal load balancer for staff and IITH systems. Each application VM runs the web, API and ClamAV containers. PostgreSQL runs natively on a primary VM with a hot standby fed by streaming replication. Documents are on an NFS share mounted by both application VMs. Losing one application VM leaves the service running; losing the database primary is handled by promoting the standby.

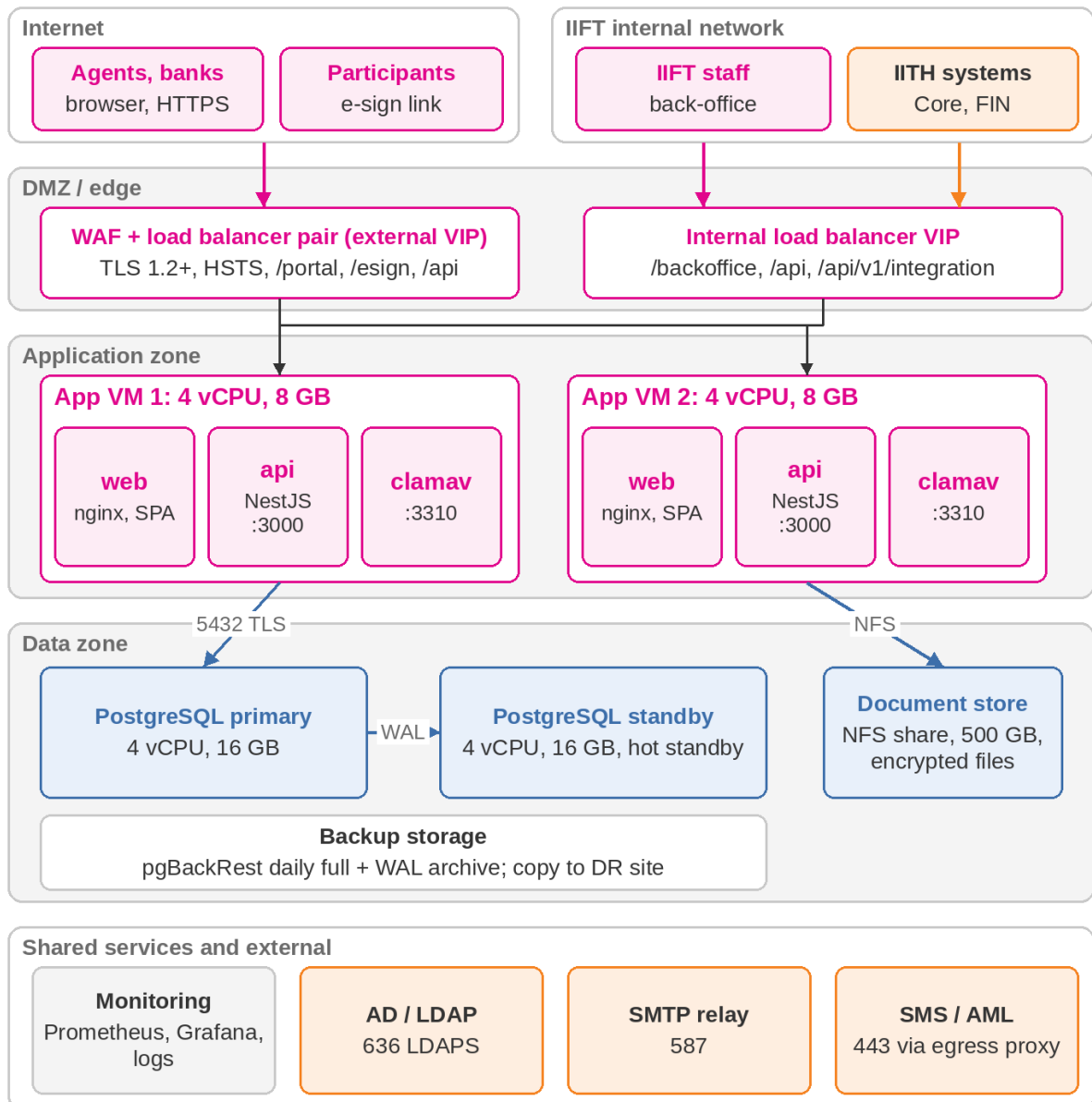
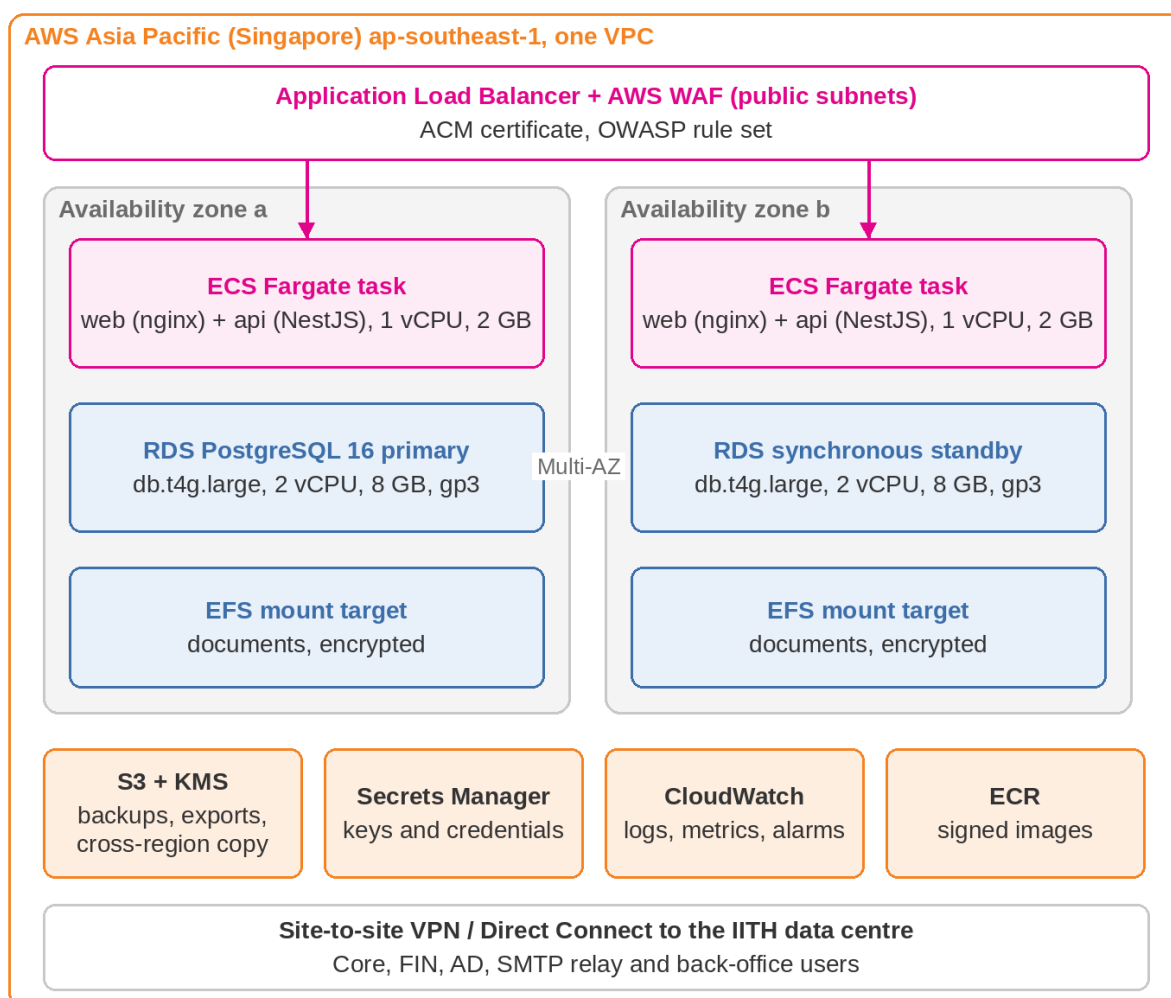


Figure 9: Option A: on-premise deployment

9.3 Option B: cloud

The same images run on AWS Asia Pacific (Singapore) or Azure Southeast Asia, subject to IIFT's data-residency decision and AMBD outsourcing and cloud notification.



Azure equivalent: Application Gateway + WAF, AKS or Container Apps, Azure Database for PostgreSQL flexible server (zone-redundant HA), Azure Files, Blob Storage, Key Vault, Azure Monitor.

Figure 10: Option B: AWS reference deployment

Component	AWS (Singapore)	Azure (Southeast Asia)
Edge	Application Load Balancer, AWS WAF, ACM certificate	Application Gateway with WAF
Containers	ECS Fargate, 2 tasks in 2 AZs	Azure Container Apps or AKS, 2 replicas in 2 zones
Database	RDS for PostgreSQL 16, Multi-AZ	Azure Database for PostgreSQL flexible server, zone-redundant HA
Documents	Amazon EFS, encrypted, mounted at /data/documents	Azure Files, encrypted
Backups and exports	S3 with SSE-KMS, cross-region copy	Blob Storage, geo-redundant
Secrets	Secrets Manager and KMS	Key Vault
Monitoring	CloudWatch logs, metrics, alarms	Azure Monitor, Log Analytics
Connectivity	Site-to-site VPN or Direct Connect to IITH	VPN Gateway or ExpressRoute

Table 22: Cloud services

9.4 Option C: hybrid

The back-office, API, database and documents stay in the IITH data centre; only the web tier for internet users (nginx with the SPA) runs in the DMZ or at a cloud edge and forwards /api calls over a private encrypted link. Personal data never leaves the data centre.

9.5 Disaster recovery

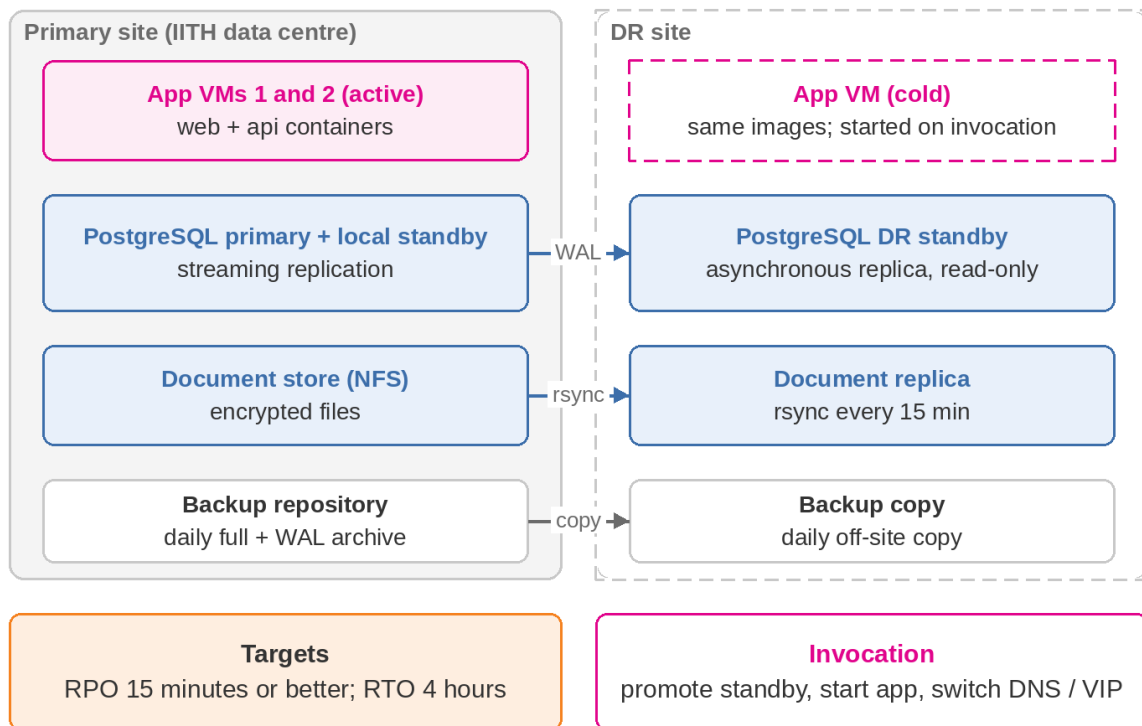


Figure 11: Disaster recovery topology

The DR site holds an asynchronous PostgreSQL replica, a copy of the document store refreshed every 15 minutes, copies of the backups, and a cold application VM with the current images. Invocation promotes the replica, mounts the document copy, starts the containers with DR configuration and moves DNS or the load-balancer VIP. The Production Support Handover gives the step-by-step procedure and the test schedule.

9.6 Container images

Image	Source	Notes
salesverse-api (runtime)	apps/api/Dockerfile, target runtime	node:22-bookworm-slim; production dependencies only; runs as user salesverse under tini; HEALTHCHECK on /health/ready; port 3000; documents at /data/documents
salesverse-api (migrate)	apps/api/Dockerfile, target migrate	One-off job: prisma migrate deploy, then the reference-data seed (idempotent)
salesverse-web	apps/web/Dockerfile	Vite build served by nginxinc/nginx-unprivileged 1.29 on port 8080 with security headers, CSP, rate limiting and /api proxy
clamav	clamav/clamav:stable	clamd for upload scanning, port 3310, internal only
PostgreSQL 16	postgres:16-alpine (DEV, SIT, UAT); OS packages on the PROD database VMs	Streaming replication and pgBackRest on PROD

Table 23: Images

Docker Compose (deploy/docker-compose.yml) runs the full stack on one host for DEV, SIT and UAT, and the web, API and ClamAV services on each production application VM, with the database on dedicated VMs. Kubernetes or ECS manifests are produced during implementation if IIFT selects a cloud option.

9.7 Build and release pipeline

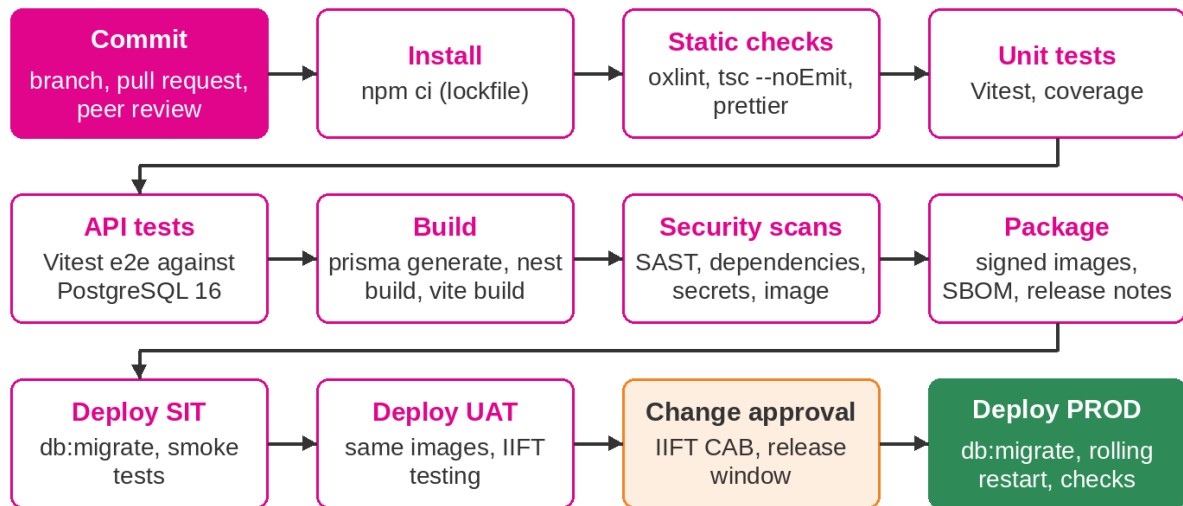


Figure 12: Build and release pipeline

Stage	Activities	Status
Verify	npm ci; prisma generate; prettier check; oxlint; tsc type check; unit tests; end-to-end API tests against a PostgreSQL 16 service; production build; npm audit (moderate and above)	Running (.github/workflows/ci.yml)
Images	Build API and web images; Trivy scan fails on unfixed CRITICAL or HIGH findings	Running
Static analysis and secrets	SAST rules and secret scanning on every pull request	Delivered during implementation
Package	Tag images with the release version, sign them, attach SBOM and release notes, push to the IITH registry	Delivered during implementation
Deploy SIT and UAT	Run the migrate job, roll out containers, smoke tests	Delivered during implementation
Deploy PROD	After IIFT change approval, in the agreed window, same images as UAT	Delivered during implementation

Table 24: Pipeline stages

Database changes follow expand-and-contract: a release first adds columns or tables that the previous version ignores, and removes old structures only in a later release. The previous images therefore run on the new schema, so an application rollback does not need a database restore.

10. Infrastructure Sizing

10.1 Load assumptions

Appendix 1 lists 26 named users: 12 bank portal users, 5 IIFT back-office users, 5 IIFT portal users and 4 agent portal users. The design target is 100 concurrent and 500 named users without redesign. Appendix 2 gives the policy volumes below.

Product	Line	2025	2026 to July	In 5 years
Financing Takaful Plan – Hire Purchase (FTP-HP)	Mortgage Takaful	59	16	75
Financing Takaful Plan – Non-Participating (FTP-NP)	Mortgage Takaful	158	65	205
Property Financing Takaful (PFT)	Mortgage Takaful	88	60	115
Personal Home Assistant Takaful (PHA)	Annual	102	41	132
Professional Takaful Plan (PRO)	Annual	40	19	52
Khairat Takaful Plan (KHR)	Annual	31	16	40
Overseas Student Assist Takaful (OSA)	Annual	7	2	9
Total		485	219	628

Table 25: Appendix 2 policy volumes per year

Growing from 485 policies in 2025 to 628 a year in five years is 5.3% a year. The sizing uses that rate from go-live: 2027: 538, 2028: 566, 2029: 596, 2030: 628, 2031: 661. Every figure is then multiplied by 10 for headroom, which also covers the target of 100 concurrent users. At this scale the limiting factor is availability, not capacity.

10.2 On-premise servers

Server	Qty	vCPU	RAM	Storage	Software
Production					
WAF / load balancer	2 (or existing)	2	4 GB	–	IITH appliance or HAProxy/nginx pair
Application VM	2	4	8 GB	100 GB	Linux, Docker Engine; web, api, clamav containers
Database VM	2	4	16 GB	200 GB SSD	PostgreSQL 16 primary and hot standby, pgBackRest
Document store	1 share	–	–	500 GB	NFS export, encrypted volume
Backup repository	1	–	–	1 TB	pgBackRest repository, document backups
Monitoring	1 (or existing)	2	4 GB	100 GB	Prometheus, Grafana, log collector
Non-production					
SIT	1	4	16 GB	200 GB	Compose: web, api, clamav, PostgreSQL
UAT	1	4	16 GB	200 GB	Compose: web, api, clamav, PostgreSQL
DR site					
Application VM	1	4	8 GB	100 GB	Cold standby, same images
Database VM	1	4	16 GB	200 GB SSD	Asynchronous replica
Document and backup copies	–	–	–	500 GB + 1 TB	rsync target, backup copy

Table 26: On-premise sizing (Option A)

Operating system: Red Hat Enterprise Linux 9 or Ubuntu 24.04 LTS, hardened to the IITH baseline. Each application VM has spare capacity for a second API container if load grows. ClamAV needs about 1.5 GB of memory for its signature database, which is included.

10.3 Cloud services

Service	Purpose	Indicative B\$ / month
Container compute (2 tasks across 2 availability zones)	Portal, back-office and API	B\$ 180
Managed PostgreSQL, Multi-AZ, with storage and automated backups	Database and PITR	B\$ 330
Application load balancer and web application firewall	TLS termination, OWASP rules	B\$ 120
NAT gateway and data transfer	Outbound integration and updates	B\$ 120
Object storage (SSE-KMS) and cross-region snapshot copy	Documents, backups, DR copies	B\$ 60
Logging, metrics and alarms	Monitoring and alerting	B\$ 60
Secrets and key management	Credentials and encryption keys	B\$ 20
Scaled-down UAT environment	Non-production testing	B\$ 60
Total per month		B\$ 950
Total per year		B\$ 11,400

Table 27: Option B indicative cost (pass-through)

Component	Size	Notes
ECS Fargate tasks	2 × (1 vCPU, 2 GB)	web and api containers per task; scale to 4 tasks if needed
RDS for PostgreSQL	db.t4g.large (2 vCPU, 8 GB), Multi-AZ	100 GB gp3, 35-day automated backups and PITR
Amazon EFS	Up to 100 GB in five years	Encrypted, lifecycle to infrequent access
S3	About 200 GB	Backups and exports, SSE-KMS, cross-region copy
ClamAV	Sidecar per task (1 GB)	Or a small shared service
UAT	1 task and a single-AZ db.t4g.medium	Scaled down, stopped outside test periods

Table 28: Cloud sizing

10.4 Database sizing

Rows per issued policy were estimated from how the application writes data: quotations not taken up, policy history, documents, payments, approvals, notifications, outbox messages, integration log entries and audit rows. Index overhead is added at 60%.

Table	Rows per policy	Bytes per row	Basis
policy	2.5	2,600	issued policy plus 1.5 quotations not taken up
policy_event	10	260	history entries
nominee	1.5	300	
participant	0.8	850	some participants hold several policies
aml_screening	1	900	screening on registration and name change
signature_request	0.5	420	
document	9	420	metadata only; files counted separately
payment	0.5	420	bulk payments cover about two policies
payment_allocation	1.1	130	
receipt	1.1	220	
commission	1	220	
claim	0.05	900	
approval_request	0.8	1,600	referrals, payment verification, endorsements
approval_action	2	260	
notification	14	900	in-app, e-mail and SMS
outbox_message	3.5	1,000	
integration_log	18	420	outbound, inbound, e-mail and SMS calls

Table	Rows per policy	Bytes per row	Basis
audit_log	35	1,500	before and after values

Table 29: Sizing basis

Year	Policies	Cumulative	Growth	Database size	×10 headroom
2027	538	538	89 MB	89 MB	0.9 GB
2028	566	1,104	93 MB	182 MB	1.8 GB
2029	596	1,700	97 MB	280 MB	2.7 GB
2030	628	2,328	102 MB	382 MB	3.7 GB
2031	661	2,989	107 MB	488 MB	4.8 GB

Table 30: Database growth

After five years the database holds about 488 MB, or 4.8 GB at ten times the volume. The 200 GB database volume therefore leaves room for WAL, temporary files, local backups and growth well beyond the contract. PostgreSQL settings: shared_buffers 4 GB, effective_cache_size 12 GB, max_connections 100 (two API instances use at most 30), wal_level replica, archive_timeout 300 seconds.

10.5 Document storage

Item	Files	Average size
Uploaded supporting documents (IC, proposal form, PDS, product documents)	5	0.40 MB
System-generated e-Policy schedule and e-Receipt (PDF)	2	0.05 MB
Signatures (PNG)	2	0.02 MB
Payment proof (shared by about two policies)	0.5	0.50 MB
Per issued policy		2.39 MB

Table 31: Document volume per policy

Year	Added	Cumulative	×10 headroom
2027	1.4 GB	1.4 GB	14 GB
2028	1.4 GB	2.8 GB	28 GB
2029	1.5 GB	4.3 GB	43 GB
2030	1.6 GB	5.9 GB	59 GB
2031	1.7 GB	7.6 GB	76 GB

Table 32: Document store growth

Including 120 MB a year for EOD reports, FIN files, scheduled reports and agent, claim and issue documents, the store reaches about 8 GB in five years and 76 GB with headroom, inside the 500 GB share.

10.6 Backup, RPO and RTO

Item	Design
Database backups	pgBackRest full backup daily at 02:00, after EOD and the overnight jobs; continuous WAL archiving with archive_timeout 300 seconds
Document backups	Daily incremental, weekly full
Retention	35 days online; monthly copies kept 12 months; yearly copies per IIFT policy
Off-site	Daily copy of backups to the DR site; backups encrypted
RPO	15 minutes or better: streaming replication to the standby is near-real-time, and archived WAL limits loss to 5 minutes if both database nodes are lost

Item	Design
RTO	4 hours for a full site loss; under 30 minutes for loss of an application VM (none, service continues) or the database primary (promote standby)
Backup storage need	About 150 GB at year 5 (database fulls and WAL, document backups, monthly copies); 1 TB provisioned
Verification	Quarterly restore test to a scratch instance with reconciliation of counts; yearly DR test

Table 33: Backup and recovery design

10.7 Network ports and firewall rules

Source	Destination	Port	Purpose
Internet	External WAF / load balancer	443/tcp	HTTPS for /portal, /esign, /api (80 redirects)
IIFT internal network	Internal load balancer	443/tcp	HTTPS for /backoffice and /api
Core and FIN hosts	Internal load balancer	443/tcp	/api/v1/integration (IP allow-list)
Load balancers	App VMs (web container)	8080/tcp	HTTP inside the zone, or TLS re-encryption if IITH policy requires
Web container	API container	3000/tcp	Same host network; not exposed outside the VM
Load balancers, monitoring	App VMs	3000/tcp	/health/live, /health/ready, /metrics (bearer token); published on the VM's internal interface by the production compose override
App VMs	PostgreSQL VIP / primary	5432/tcp	TLS, application role
App VMs	Document store (NFS)	2049/tcp	NFSv4, export restricted to app VMs
App VMs	SMTP relay	587/tcp	STARTTLS
App VMs	Active Directory	636/tcp	LDAPS
App VMs	Core and FIN APIs	443/tcp	HTTPS
App VMs	Egress proxy to SMS gateway and AML provider	443/tcp	HTTPS, destination allow-list
DB primary	DB standby (local and DR)	5432/tcp	Streaming replication, TLS
DB VMs	Backup repository	22/tcp	pgBackRest over SSH
Primary site	DR site	22/tcp	Document rsync, backup copy
Monitoring	All VMs	9100/tcp, 9187/tcp	node_exporter and postgres_exporter
Administrators (bastion)	All VMs	22/tcp	SSH with keys, from the bastion only
All VMs	IITH NTP and DNS	123/udp, 53	Time and name resolution

Table 34: Firewall matrix (Option A)

11. Non-functional Design

Area	Target	Design measures
Performance (NFR-01)	Page load under 3 s; API p95 under 1 s; reports on 12 months of data under 10 s	Indexed list queries with pagination (max 100 rows); lazy-loaded pages and split bundles; gzip and long caching of hashed assets; exports capped and streamed to file
Capacity (NFR-02, 03)	100 concurrent, 500 named users; 10× Appendix 2 volume	Stateless API instances; connection pools of 10 per instance plus 5 for sessions
Scalability (NFR-06, 07)	Growth without redesign	Add API containers or VMs behind the load balancer; jobs stay single-runner through advisory locks; database vertical scaling
Availability (NFR-04, 05)	99.5% monthly; 99.9% in IIFT business hours	Two application VMs; hot standby database; health-checked load balancing; rolling deployment
Integrity (NFR-16)	No partial updates	Single-transaction business operations; outbox; optimistic locking
Recoverability (NFR-17 to 19)	RPO 15 minutes; RTO 4 hours	Streaming replication, WAL archiving, daily backups, DR site, tested restores
Usability and accessibility (NFR-22 to 24)	WCAG 2.1 AA; current Chrome, Edge, Firefox, Safari	Ant Design accessible components, keyboard navigation, consistent layouts in both applications
API standards (NFR-25)	REST, JSON, OpenAPI 3	/api/v1 versioning; ISO 8601 dates; documented error body; OpenAPI description in docs/api
Maintainability (NFR-20, 21)	Modular, documented, tested	11 unit test files (54 cases) and 4 end-to-end suites (29 cases) run in CI; lint, format and type checks

Table 35: Non-functional targets and design

11.1 Monitoring and health

Endpoint or signal	Content	Use
GET /health/live	{status: ok} while the process runs	Container and load-balancer liveness
GET /health/ready	Runs SELECT 1; 503 NOT_READY if the database is unavailable	Load-balancer readiness; Docker HEALTHCHECK
GET /metrics	Prometheus default process metrics prefixed iift_api_ (CPU, memory, heap, event-loop lag, GC); bearer METRICS_TOKEN	Prometheus scrape every 30 s
Integration monitor	Per-system success rate, average time, pending, retrying and dead-letter counts for 24 hours	Back-office screen and API for support
EOD history	Status, totals and files per business date	Finance and support
Application e-mails	Dead-letter alerts, AML review, SLA breaches, agency blocks	Business and support teams

Table 36: Health and monitoring signals

Prometheus alert rules (delivered during implementation) cover: readiness failing for 2 minutes; HTTP 5xx rate above 2% for 5 minutes (from nginx logs); p95 latency above 2 s; event-loop lag above 500 ms; memory above 85%; disk above 80%; replication lag above 5 minutes; last successful backup older than 26 hours; an eod_run in FAILED status or no completed run by 01:00; any outbox message in DEAD; certificate expiry within 30 days. HTTP request histograms and business gauges (outbox backlog, open SLA breaches) are added to /metrics during implementation.

11.2 Logging

The API writes one JSON line per event to standard output with pino: level, time, message and, for HTTP requests, the request id, method, URL, status and response time. The request id is the correlation id from X-Request-Id (set by nginx) or a new UUID, and is returned in the response header and in every error body, so a user's reference leads straight to the log lines. Cookies, authorization, CSRF and set-cookie headers are

redacted; health probes are not logged. Container logs are collected by the platform's log shipper (Fluent Bit to the IITH log platform, or CloudWatch) and kept 90 days online. The audit trail is separate from logs and is held in the database under the retention rules.

12. Technology Stack and Versions

Versions are the minimums declared in the package manifests at the time of writing; the lockfile pins exact versions. All components are open source under permissive licences; there are no runtime licence fees.

Component	Version	Purpose
Runtime		
Node.js	22 LTS (.nvmrc 22, engines >=22.12)	API runtime
npm	11.21.0	workspaces
API		
@nestjs/core	12.1.2	Application framework
@nestjs/schedule	12.0.2	Scheduled jobs
@nestjs/throttler	6.7.1	Rate limiting
@nestjs/swagger	12.0.2	OpenAPI
class-validator	0.15.1	DTO validation
typescript	6.0.3	Language
Data		
PostgreSQL	16	Database
prisma / @prisma/client	7.10.0	ORM and migrations
@prisma/adapters-pg, pg	8.23.1	Driver
Security		
express-session	1.19.0	Sessions
connect-pg-simple	10.0.0	Session store
@node-rs/argon2	2.2.1	Password hashing
helmet	8.3.0	Security headers
ldapts	9.2.0	Directory sign-in
Output and messaging		
pdfkit	0.20.2	PDF documents
exceljs	4.4.0	Excel exports
nodemailer	10.0.13	E-mail
Observability		
nestjs-pino / pino	10.3.1	JSON logging
prom-client	15.1.3	Metrics
Front end		
react	19.3.0	UI library
antd	6.6.5	Components
@tanstack/react-query	5.104.1	Server state
react-router	7.18.4	Routing
recharts	3.10.1	Charts
vite	7.3.6	Build tool
Quality		
vitest	4.1.11	Unit and API tests
supertest	7.3.0	HTTP tests
eslint	1.86.0	Linting
prettier	3.9.9	Formatting

Component	Version	Purpose
Delivery		
Docker images	node:22-bookworm-slim, nginx-unprivileged 1.29	Containers
GitHub Actions / GitLab CI	ci.yml	Pipeline
Trivy	0.28 action	Image scan

Table 37: Technology stack

13. Architecture Decisions

Each decision is recorded with the options considered and the reason for the choice. Changes to these decisions follow the change process and are added here.

Id	Decision	Options considered	Rationale
AD-01	Modular monolith	Microservices; modular monolith	Small user base and volume; one deployment and one database transaction per operation; module boundaries keep a later split possible.
AD-02	PostgreSQL 16	PostgreSQL; MongoDB; SQL Server	Relational, financial data with ACID across tables; no licence fee; mature replication and PITR; JSONB where flexibility is needed (section 6.2).
AD-03	Prisma 7 with the pg driver adapter	Prisma; TypeORM; Knex; hand-written SQL	Typed queries generated from one schema; parameterised by construction; versioned SQL migrations reviewed in code review.
AD-04	Server-side sessions	Sessions in PostgreSQL; JWT in browser storage	Revocable at once (sign-out, logout, deactivation, password change); no tokens readable by scripts; no extra session store.
AD-05	Synchroniser CSRF token with SameSite=Strict	Synchroniser token; double-submit cookie; SameSite only	Token bound to the server session and checked in constant time; SameSite gives a second layer.
AD-06	Argon2id	Argon2id; bcrypt; PBKDF2	Current OWASP recommendation; memory-hard; native implementation without build tools.
AD-07	Application-level field encryption with blind index	Disk encryption only; pgcrypto; application encryption	Keys never reach the database server; database administrators and backups see ciphertext; exact search and uniqueness still work.
AD-08	Transactional outbox	Synchronous calls; message broker; outbox table	A message exists exactly when the business change commits; retries survive restarts; no broker to run for this volume.
AD-09	In-process jobs with advisory locks	Separate scheduler; Redis queue; in-process with locks	No extra component; one runner across replicas; lock released if a process dies.
AD-10	Encrypted files on a mounted volume	Database BLOBs; object storage; encrypted volume	Keeps the database small; works on NFS on-premise and EFS or Azure Files in the cloud; S3 adapter possible later.
AD-11	React SPA with Ant Design	React SPA; server-rendered pages; Angular	Large talent pool; accessible component library; one code base for portal and back-office.
AD-12	Business parameters in the database	Environment variables; database parameters	Operations staff change policy values without a release, within ranges, with audit.
AD-13	Generic workflow engine with handlers	Per-feature approval code; BPMN engine; generic engine	Eight approval types share one inbox, rule set and history; a BPMN engine would add a product to operate.
AD-14	Watch-list screening built in, external provider optional	External only; built in only; both	Compliance can screen from day one with UN and local lists; a commercial provider can be added by configuration; an unavailable provider forces manual review.
AD-15	PostgreSQL sequences for reference numbers	Sequences; counter table with row locks	No lock contention across replicas; gaps are acceptable for these references.
AD-16	Optimistic locking	Pessimistic locks; version column	Short web transactions; clear STALE_RECORD message instead of blocking users.
AD-17	Simulated and live integration modes	Live only; switchable mode	Build and test end to end before IITH endpoints exist; production start-up checks the live settings.
AD-18	Manual database promotion on PROD	Automatic failover; manual promotion	Two database nodes cannot form a safe quorum; a runbook promotion within the 4-hour RTO avoids split-brain.
AD-19	JSON logs and Prometheus metrics	Vendor APM agent; open formats	Works with IITH's existing log and monitoring tools; no agent licence.

Table 38: Architecture decision log

Appendix A: API Operations

All 149 operations on 128 paths in docs/api/openapi.json, grouped by tag. Request and response schemas are in the OpenAPI file.

Method	Path
Authentication	
POST	/api/v1/auth/login
POST	/api/v1/auth/logout
GET	/api/v1/auth/me
POST	/api/v1/auth/change-password
Back-office: AML/KYC	
GET	/api/v1/backoffice/aml/cases
POST	/api/v1/backoffice/aml/cases/{id}/review
GET	/api/v1/backoffice/aml/watchlist
POST	/api/v1/backoffice/aml/watchlist
PUT	/api/v1/backoffice/aml/watchlist/{id}/active
POST	/api/v1/backoffice/aml/watchlist/import
Back-office: Agencies	
GET	/api/v1/backoffice/agencies
POST	/api/v1/backoffice/agencies
GET	/api/v1/backoffice/agencies/options
GET	/api/v1/backoffice/agencies/{id}
PUT	/api/v1/backoffice/agencies/{id}
POST	/api/v1/backoffice/agencies/{id}/lift-block
Back-office: Agents	
GET	/api/v1/backoffice/agents
POST	/api/v1/backoffice/agents
GET	/api/v1/backoffice/agents/{id}
POST	/api/v1/backoffice/agents/{id}/update-requests
POST	/api/v1/backoffice/agents/{id}/status-requests
Back-office: Approvals	
GET	/api/v1/backoffice/approvals/inbox
GET	/api/v1/backoffice/approvals
GET	/api/v1/backoffice/approvals/{id}
POST	/api/v1/backoffice/approvals/{id}/approve
POST	/api/v1/backoffice/approvals/{id}/reject
POST	/api/v1/backoffice/approvals/{id}/withdraw
Back-office: Audit	
GET	/api/v1/backoffice/audit
GET	/api/v1/backoffice/audit/facets
Back-office: Billing	
GET	/api/v1/backoffice/billing/payments
GET	/api/v1/backoffice/billing/payments/{id}
GET	/api/v1/backoffice/billing/commissions
Back-office: Claims	
GET	/api/v1/backoffice/claims
GET	/api/v1/backoffice/claims/{id}
PUT	/api/v1/backoffice/claims/{id}/status
Back-office: Configuration	
GET	/api/v1/backoffice/config/parameters
PUT	/api/v1/backoffice/config/parameters/{key}
GET	/api/v1/backoffice/config/codes
POST	/api/v1/backoffice/config/codes
GET	/api/v1/backoffice/config/code-categories
PATCH	/api/v1/backoffice/config/codes/{id}
Back-office: Dashboard	
GET	/api/v1/backoffice/dashboard

Method	Path
Back-office: Documents	
GET	/api/v1/backoffice/documents
POST	/api/v1/backoffice/documents/{id}/review
Back-office: End of day	
GET	/api/v1/backoffice/eod
POST	/api/v1/backoffice/eod
Back-office: Integration	
GET	/api/v1/backoffice/integration/summary
GET	/api/v1/backoffice/integration/outbox
POST	/api/v1/backoffice/integration/outbox/{id}/retry
GET	/api/v1/backoffice/integration/logs
GET	/api/v1/backoffice/integration/reconciliations
POST	/api/v1/backoffice/integration/reconciliations
Back-office: Issues	
GET	/api/v1/backoffice/issues/assignees
PUT	/api/v1/backoffice/issues/{id}/assignment
PUT	/api/v1/backoffice/issues/{id}/priority
Back-office: Participants	
GET	/api/v1/backoffice/participants
GET	/api/v1/backoffice/participants/{id}
POST	/api/v1/backoffice/participants/{id}/update-requests
Back-office: Policies	
GET	/api/v1/backoffice/policies
GET	/api/v1/backoffice/policies/{id}
POST	/api/v1/backoffice/policies/{id}/email-documents
Back-office: Products	
GET	/api/v1/backoffice/products
PUT	/api/v1/backoffice/products/{id}
Back-office: Reports	
GET	/api/v1/backoffice/reports
GET	/api/v1/backoffice/reports/schedules
POST	/api/v1/backoffice/reports/schedules
PUT	/api/v1/backoffice/reports/schedules/{id}/active
GET	/api/v1/backoffice/reports/{code}
GET	/api/v1/backoffice/reports/{code}/export
Back-office: Roles	
GET	/api/v1/backoffice/roles
POST	/api/v1/backoffice/roles
GET	/api/v1/backoffice/roles/permissions
PUT	/api/v1/backoffice/roles/{id}
DELETE	/api/v1/backoffice/roles/{id}
Back-office: Users	
GET	/api/v1/backoffice/users
POST	/api/v1/backoffice/users
GET	/api/v1/backoffice/users/{id}
PATCH	/api/v1/backoffice/users/{id}
PUT	/api/v1/backoffice/users/{id}/status
POST	/api/v1/backoffice/users/{id}/unlock
POST	/api/v1/backoffice/users/{id}/reset-password
GET	/api/v1/backoffice/role-options
Back-office: Workflow configuration	
GET	/api/v1/backoffice/workflows
PUT	/api/v1/backoffice/workflows/{type}
Common: Documents	
POST	/api/v1/common/documents
GET	/api/v1/common/documents
GET	/api/v1/common/documents/{id}/content

Method	Path
Common: Issues	
GET	/api/v1/common/issues
POST	/api/v1/common/issues
GET	/api/v1/common/issues/{id}
POST	/api/v1/common/issues/{id}/comments
PUT	/api/v1/common/issues/{id}/status
Common: Master data	
GET	/api/v1/common/codes
Common: Notifications	
GET	/api/v1/common/notifications
GET	/api/v1/common/notifications/unread-count
POST	/api/v1/common/notifications/{id}/read
POST	/api/v1/common/notifications/read-all
Common: Products	
GET	/api/v1/common/products
GET	/api/v1/common/products/{id}
Integration (system-to-system)	
PUT	/api/v1/integration/agents/{agentCode}/status
GET	/api/v1/integration/policies
POST	/api/v1/integration/commissions/paid
Portal: Billing	
GET	/api/v1/portal/billing/outstanding
GET	/api/v1/portal/billing/payments
POST	/api/v1/portal/billing/payments
GET	/api/v1/portal/billing/payments/{id}
GET	/api/v1/portal/billing/commissions
Portal: Claims	
GET	/api/v1/portal/claims
POST	/api/v1/portal/claims
GET	/api/v1/portal/claims/validate-policy
GET	/api/v1/portal/claims/{id}
Portal: Dashboard	
GET	/api/v1/portal/dashboard
Portal: Participants	
GET	/api/v1/portal/participants
POST	/api/v1/portal/participants
GET	/api/v1/portal/participants/lookup
GET	/api/v1/portal/participants/{id}
POST	/api/v1/portal/participants/{id}/update-requests
Portal: Profile & agency	
GET	/api/v1/portal/profile
POST	/api/v1/portal/profile/update-requests
GET	/api/v1/portal/agency
GET	/api/v1/portal/hierarchy
GET	/api/v1/portal/agents
POST	/api/v1/portal/agents
GET	/api/v1/portal/agents/{id}
Portal: Quotations & policies	
GET	/api/v1/portal/policies
POST	/api/v1/portal/policies
GET	/api/v1/portal/policies/renewals-due
GET	/api/v1/portal/policies/{id}
PUT	/api/v1/portal/policies/{id}
POST	/api/v1/portal/policies/calculate
PUT	/api/v1/portal/policies/{id}/questionnaire
PUT	/api/v1/portal/policies/{id}/nominees
POST	/api/v1/portal/policies/{id}/submit
POST	/api/v1/portal/policies/{id}/discard

Method	Path
POST	/api/v1/portal/policies/{id}/reopen
POST	/api/v1/portal/policies/{id}/renew
POST	/api/v1/portal/policies/{id}/endorsements
POST	/api/v1/portal/policies/{id}/cancellations
POST	/api/v1/portal/policies/{id}/email-documents
Portal: Reports	
GET	/api/v1/portal/reports
GET	/api/v1/portal/reports/{code}
GET	/api/v1/portal/reports/{code}/export
Portal: Requests	
GET	/api/v1/portal/requests
GET	/api/v1/portal/requests/{id}
POST	/api/v1/portal/requests/{id}/withdraw
Portal: e-Signature	
POST	/api/v1/portal/policies/{id}/signatures
POST	/api/v1/portal/policies/{id}/signatures/link
Public: e-Signature	
GET	/api/v1/public/esign/{token}
POST	/api/v1/public/esign/{token}

Appendix B: Glossary

Term	Meaning
Advisory lock	PostgreSQL lock identified by a number, used here so one API instance runs each job
AMBD	Autoriti Monetari Brunei Darussalam, the financial regulator
Blind index	Keyed hash of a value stored next to its ciphertext so the value can be matched exactly without decrypting
CSRF	Cross-site request forgery
Dead-letter	State of a message that will not be retried automatically
EOD	End-of-day processing
FIN	IITH financial system
Idempotency key	Identifier that lets a receiver ignore a repeated delivery
Maker-checker	Control where one person raises a change and another approves it
Outbox	Table of messages written with the business change and delivered afterwards
PDPO	Personal Data Protection Order 2025 (Brunei)
PITR	Point-in-time recovery
RPO / RTO	Recovery point objective (data loss) / recovery time objective (downtime)
SPA	Single-page application
Tabarru'	Donation portion of the contribution to the participants' risk fund
Wakalah	Agency fee charged by the takaful operator